

מסמך ארכיטקטורה, מנגנונים ומילון מחלקות מורחב

UI_alfa - Comprehensive System Manual

מדריך טכני מקיף: ניתוח מנגנוני הליבה, קווי אשראי, ניתוב AI, ומילון מפורט של
100% ממחלקות ה-Backend וה-Frontend (64 מחלקות + 59 Java קבצי
(Frontend

סה"כ מחלקות Backend (Java): 64 מחלקות

סה"כ רכיבים וקבצי Frontend: 59 קבצים

אינטגרציות: MetaTrader 5 Bridge, Binance API, Yahoo Finance, Finnhub, AI Neural Network Predictor

פורמט נקרא: עברית ברורה (RTL) מול קוד וסמלים באנגלית (LTR)

תאריך עדכון: יולי 2026

תוכן עניינים מפורט

1. פרק 1: סקירה ארכיטקטונית ומנגנוני הליבה
2. פרק 2: מנגנון הזדהות, אבטחה וכניסה - משתמש רגיל vs. משתמש דמו
3. פרק 3: מנגנון קנייה ומכירה של נכסים וביצוע הוראות
4. פרק 4: מנגנון הקרדיט ליין, ניהול סיכונים ומנוע הנזלה כפויה
5. פרק 5: מנגנון המאצ'ינג, ניתוב AI וגשר MetaTrader 5
6. פרק 6: מילון מפורט - REST Controllers (7 מחלקות)
7. פרק 7: מילון מפורט - Core Services & Integrations (15 מחלקות)
8. פרק 8: מילון מפורט - Configs, Security & Setup (6 מחלקות)
9. פרק 9: מילון מפורט - JPA Entities (24 מחלקות)
10. פרק 10: מילון מפורט - JPA Repositories (13 מחלקות)
11. פרק 11: מילון מפורט - Frontend Components, Hooks & Scripts (59 קבצים)

פרק 1: סקירה ארכיטקטונית ומנגנוני הליבה

מערכת **UI_alfa** הינה פלטפורמת מסחר וברוקראז' אלגוריתמית מקצה לקצה. המערכת כוללת מנוע מסחר בזמן אמת, מנוע אשראי ומינוף (מנגנון Margin Loan בסגנון Bybit), מנוע ניתוב מונחה בינה מלאכותית (NN Predictor), ואינטגרציות מול שרת MetaTrader 5 ובורסות קריפטו ומניות (Binance API, Yahoo Finance, Finnhub).

ארכיטקטורה מודולרית: ה-Backend תוכנן ב-Java 17 וב-Spring Boot 3, תוך הפרדה מלאה בין ה-Controllers, ה-Entities, ה-Repositories וה-Services. ה-Frontend מבוסס React ב-Vite עם Tailwind CSS.

פרק 2: מנגנון הזדהות, אבטחה וכניסה - משתמש רגיל vs. משתמש דמו

המערכת מפרידה בצורה מלאה בין שני סוגי משתמשים:

- **משתמש רגיל (Real Account):** נדרש ברישום ב-AuthApiController.java, הצפנת סיסמה ב-BCrypt, והגשת מסמכי אימות זהות ב-KycCase.java.
- **משתמש דמו (Demo Simulators):** מועלה אוטומטית בהפעלת השרת ב-AuthBootstrap.java עם 4 תרחישי דמו (High Risk Trader, VIP Trader, Hedging Trader, Default Demo).

פרק 3: מנגנון קנייה ומכירה של נכסים וביצוע הוראות

מנוע המסחר ב-OrderExecutionService.java מנהל הוראות MARKET, LIMIT ו-STOP.

$$\text{Margin Required} = (\text{Execution Price} \times \text{Quantity} \times \text{Contract Size}) / \text{Leverage}$$

פרק 4: מנגנון הקרדיט לייך, ניהול סיכונים ומנוע הנזלה

כפוייה

מנגנון האשראי ב-MarginLoanService.java מעניק אשראי אוטומטי (tryCoverShortfall), פורע חובות מרווחים (repaySettlementOrBorrow), מחייב ריבית יומית (0.5%/day), ומנזיל חשבונות שרמת ה-Margin Level שלהם יורדת מתחת ל-100%.

$$\text{Margin Level} = \text{Live Equity} / \text{Borrowed Balance}$$

פרק 5: מנגנון המאצ'ינג, ניתוב AI וגשר MetaTrader 5

מנוע הניתוב משלב את NNPredictorClient.java להכרעת B-Book vs A-Book, וגשר MT5 Bridge ב- MT5IntegrationService.java ו-MT5ConnectionManager.java.

פרק 6: מילון מפורט - REST Controllers (7 מחלקות)

בקר API REST

com.brokerui.broker.MT5BridgeController

נתיב קובץ: MT5_Infrastructure_v2/Backend/MT5BridgeController.java | סוג: class | אנוטציות: @RestController, @RequestBody, @GetMapping, @RequestParam, @PostMapping, @RequestMapping | שורות קוד: 52

תפקיד המחלקה במערכת: ה-Controller מאפשר חשיפת REST Endpoints לתקשורת מול ה-Frontend או מנהלי המערכת. מנהל את קבלת הבקשות, אימות ההרשאות, והעברת הלוגיקה לשכבת ה-Services.

פונקציות ושיטות במחלקה: `checkSymbol()`, `getCandles()`, `trade()`

תיאור ודוקומנטציה: מחלקה לניהול לוגיקה עסקית, אבטחה, אשראי או נתונים במערכת.

```

package com.brokerui.broker;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.Map;

@RestController
@RequestMapping("/api/mt5-bridge")
public class MT5BridgeController {

    private final MT5BridgeService mt5Service;

    public MT5BridgeController(MT5BridgeService mt5Service) {
        this.mt5Service = mt5Service;
    }

    @GetMapping("/candles")
    public ResponseEntity<?> getCandles(
        @RequestParam String symbol,
        @RequestParam(defaultValue = "1h") String timeframe,
        @RequestParam(defaultValue = "100") int count) {
        try {
            Candle[] candles = mt5Service.getCandles(symbol, timeframe, count);
            return ResponseEntity.ok(candles);
        } catch (Exception e) {
            return ResponseEntity.status(500).body(Map.of("error", e.getMessage()));
        }
    }

    @PostMapping("/trade")
    public ResponseEntity<?> trade(@RequestBody Map<String, Object> body) {
        String symbol = (String) body.get("symbol");
        String action = (String) body.get("action");
        double lot = Double.parseDouble(body.get("lot").toString());
        double tp = body.containsKey("tp") ? Double.parseDouble(body.get("tp").toString()) : 0;
        double sl = body.containsKey("sl") ? Double.parseDouble(body.get("sl").toString()) : 0;

        String result = mt5Service.executeTrade(symbol, action, lot, tp, sl);
        if (result.startsWith("SUCCESS")) {
            return ResponseEntity.ok(Map.of("message", result));
        } else {
            return ResponseEntity.status(500).body(Map.of("error", result));
        }
    }

    @GetMapping("/check-symbol")
    public ResponseEntity<?> checkSymbol(@RequestParam String symbol) {
        boolean available = mt5Service.isMT5Symbol(symbol);
        return ResponseEntity.ok(Map.of("symbol", symbol, "available_in_mt5", available));
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/admin/AdminTradeController.java | סוג: class
 | אנוטציות:
 @param, @org, @CrossOrigin, @RestController, @RequestBody, @GetMapping, @PathVariable, @return,
 @PostMapping, @RequestMapping
 | שורות קוד: 386

תפקיד המחלקה במערכת: ה-Controller מאפשר חשיפת REST Endpoints לתקשורת מול ה-Frontend או מנהלי המערכת. מנהל את קבלת הבקשות, אימות ההרשאות, והעברת הלוגיקה לשכבת ה-Services.

פונקציות ושיטות במחלקה: `getKyc()`, `getAuditLog()`, `AccountDto()`, `UpdateLoanRequest()`, `AuditDto()`,
`TransactionDto()`, `getAllTrades()`, `TradeDto()`, `KycDto()`, `getMarginLoans()`, `MarginLedgerDto()`,
`NotifDto()`, `getAccounts()`, `getTransactions()`, `getNotifications()`, `UpdateBalanceRequest()`

תיאור ודוקומנטציה: REST controller for administrator functions, including server control, MT5 bridge connection management, viewing system-wide accounts, trades, audits, transactions, and updating account balances.

```

package com.brokerui.admin;

import com.brokerui.broker.BrokerOrder;
import com.brokerui.broker.BrokerOrderRepository;
import com.brokerui.broker.Position;
import com.brokerui.broker.PositionRepository;
import com.brokerui.broker.TradingAccountRepository;
import com.brokerui.broker.AccountTransactionRepository;
import com.brokerui.broker.KycCaseRepository;
import com.brokerui.broker.NotificationRepository;
import com.brokerui.broker.AuditLogRepository;
import com.brokerui.broker.MT5ConnectionManager;
import com.brokerui.broker.MarginLoanLedger;
import com.brokerui.broker.MarginLoanLedgerRepository;
import com.brokerui.broker.MarginLoanService;
import org.springframework.context.ApplicationContext;
import org.springframework.boot.SpringApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.CrossOrigin;

import java.math.BigDecimal;
import java.time.Instant;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

/**
 * REST controller for administrator functions, including server control,
 * MT5 bridge connection management, viewing system-wide accounts, trades,
 * audits, transactions, and updating account balances.
 */
@RestController
@CrossOrigin
@RequestMapping("/api/admin")
public class AdminTradeController {

    private final BrokerOrderRepository orderRepo;
    private final PositionRepository positionRepo;
    private final TradingAccountRepository accountRepo;
    private final AccountTransactionRepository transactionRepo;
    private final KycCaseRepository kycRepo;
    private final NotificationRepository notifRepo;
    private final AuditLogRepository auditRepo;
    private final MT5ConnectionManager mt5ConnectionManager;
    private final ApplicationContext applicationContext;

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/admin/AdminUserController.java | סוג: class |
אנוטציות:

@Valid, @param, @Transactional, @throws, @CrossOrigin, @RestController, @RequestBody,
@GetMapping, @PathVariable, @return, @PostMapping, @RequestMapping

| שורות קוד: 141

תפקיד המחלקה במערכת: ה-Controller מאפשר חשיפת REST Endpoints לתקשורת מול ה-Frontend או מנהלי המערכת. מנהל את קבלת הבקשות, אימות ההרשאות, והעברת הלוגיקה לשכבת ה-Services.

פונקציות ושיטות במחלקה: `listUsers()`, `UpdateUserRequest()`, `BanRequest()`

תיאור ודוקומנטציה: REST controller for managing users within the administration panel. Supports list, ban, unban, delete, and update operations.

```

package com.brokerui.admin;

import com.brokerui.user.AppUser;
import com.brokerui.user.AppUserRepository;
import com.brokerui.user.UserDto;
import jakarta.validation.Valid;
import java.time.Instant;
import java.util.List;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.server.ResponseStatusException;

/**
 * REST controller for managing users within the administration panel.
 * Supports list, ban, unban, delete, and update operations.
 */
@RestController
@CrossOrigin
@RequestMapping("/api/admin")
public class AdminUserController {
    private final AppUserRepository repo;

    /**
     * Constructs the AdminUserController with the user repository.
     *
     * @param repo the user repository
     */
    public AdminUserController(AppUserRepository repo) {
        this.repo = repo;
    }

    /**
     * Retrieves a list of all users in the system.
     * The goal is to provide administration overview of app users.
     *
     * @return a list of user details mapped to DTOs
     */
    @GetMapping("/users")
    public List<UserDto> listUsers() {
        return repo.findAll().stream().map(UserDto::from).toList();
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/api/HealthController.java | סוג: class
אנוטציות: @return, @GetMapping, @RestController | שורות קוד: 25

תפקיד המחלקה במערכת: ה-Controller מאפשר חשיפת REST Endpoints לתקשורת מול ה-Frontend או מנהלי המערכת. מנהל את קבלת הבקשות, אימות ההרשאות, והעברת הלוגיקה לשכבת ה-Services.

פונקציות ושיטות במחלקה: אין שיטות ציבוריות ישירות (Record/DTO)

תיאור ודוקומנטציה: Controller providing system health check endpoint.

```
package com.brokerui.api;

import java.time.Instant;
import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

/**
 * Controller providing system health check endpoint.
 */
@RestController
public class HealthController {

    /**
     * Endpoint returning system health check status and current timestamp.
     * Used by frontend polling to confirm server availability.
     *
     * @return a map indicating status and timestamp
     */
    @GetMapping("/api/health")
    public Map<String, Object> health() {
        return Map.of("ok", true, "ts", Instant.now().toString());
    }
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/auth/AuthApiController.java | סוג: class |
אנוטציות:

@Valid, @param, @Transactional, @AuthenticationPrincipal, @RestController, @RequestBody,
@GetMapping, @return, @PostMapping

| שורות קוד: 92

תפקיד המחלקה במערכת: ה-Controller מאפשר חשיפת REST Endpoints לתקשורת מול ה-Frontend או מנהלי המערכת. מנהל את קבלת הבקשות, אימות ההרשאות, והעברת הלוגיקה לשכבת ה-Services.

פונקציות ושיטות במחלקה: `me()`, `register()`

תיאור ודוקומנטציה: REST controller for handling user authentication endpoints. Handles registration, fetching current session profile, and retrieving CSRF tokens.

```

package com.brokerui.auth;

import com.brokerui.user.AppUser;
import com.brokerui.user.AppUserRepository;
import com.brokerui.user.UserDto;
import com.brokerui.user.UserRole;
import jakarta.validation.Valid;
import java.util.Locale;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.csrf.CsrfToken;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

/**
 * REST controller for handling user authentication endpoints.
 * Handles registration, fetching current session profile, and retrieving CSRF tokens.
 */
@RestController
public class AuthApiController {
    private final AppUserRepository repo;
    private final PasswordEncoder encoder;

    /**
     * Constructs the AuthApiController.
     *
     * @param repo the user accounts database repository
     * @param encoder password hashing utility encoder
     */
    public AuthApiController(AppUserRepository repo, PasswordEncoder encoder) {
        this.repo = repo;
        this.encoder = encoder;
    }

    /**
     * Retrieves the current Spring Security CSRF token.
     * The goal of this endpoint is to support JavaScript CSRF client handshakes.
     *
     * @param token the active CsrfToken populated by Spring Security
     * @return a map containing the CSRF token string value
     */
    @GetMapping("/api/auth/csrf")
    public Map<String, String> csrf(CsrfToken token) {

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/BrokerApiController.java | סוג: class
 | אנוטציות:
 @param, @returns, @Transactional, @RestController, @RequestBody, @GetMapping, @PathVariable,
 @PostMapping, @RequestMapping
 | שורות קוד: 807

תפקיד המחלקה במערכת: ה-Controller מאפשר חשיפת REST Endpoints לתקשורת מול ה-Frontend או מנהלי המערכת. מנהל את קבלת הבקשות, אימות ההרשאות, והעברת הלוגיקה לשכבת ה-Services.

פונקציות ושיטות במחלקה: `PlaceOrderRequest()`, `positions()`, `symbols()`, `cancelOrder()`, `pendingOrders()`, `history()`, `createTransaction()`, `savePreference()`, `closePosition()`, `orders()`, `overview()`, `notifications()`, `transactions()`, `placeOrder()`, `getPreferences()`

תיאור ודוקומנטציה: LONG $\text{Equity} = \text{cash balance} + \text{sum of unrealized PNL across ALL open positions}$.
 $\text{unrealized} = (\text{currentPrice} - \text{avgEntry}) \text{ qty}$ SHORT $\text{unrealized} = (\text{avgEntry} - \text{currentPrice}) \text{ qty}$ (These are already stored on each Position entity and updated on every trade.)

```

package com.brokerui.broker;

import com.brokerui.user.AppUser;
import com.brokerui.user.AppUserRepository;
import jakarta.servlet.http.HttpServletRequest;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Instant;
import java.util.List;
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.Authentication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.transaction.annotation.Transactional;

@RestController
@RequestMapping("/api/broker")
public class BrokerApiController {
    private final AppUserRepository userRepo;
    private final TradingAccountRepository accountRepo;
    private final SymbolRepository symbolRepo;
    private final PositionRepository positionRepo;
    private final BrokerOrderRepository orderRepo;
    private final TradeFillRepository fillRepo;
    private final AccountTransactionRepository txRepo;
    private final NotificationRepository notificationRepo;
    private final KycCaseRepository kycRepo;
    private final UserPreferenceRepository preferenceRepo;
    private final MT5IntegrationService mt5Service;
    private final com.brokerui.market.MarketPriceService priceService;
    private final AuditLogService auditLogService;
    private final NNPredictorClient nnPredictorClient;
    private final MarginLoanService marginLoanService;

    public BrokerApiController(
        AppUserRepository userRepo,
        TradingAccountRepository accountRepo,
        SymbolRepository symbolRepo,
        PositionRepository positionRepo,
        BrokerOrderRepository orderRepo,
        TradeFillRepository fillRepo,
        AccountTransactionRepository txRepo,
        NotificationRepository notificationRepo,
        KycCaseRepository kycRepo,

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/market/MarketController.java | סוג: class |
אנוסציות: @param, @RestController, @GetMapping, @RequestParam, @PathVariable, @return | שורות קוד:
187

תפקיד המחלקה במערכת: ה-Controller מאפשר חשיפת REST Endpoints לתקשורת מול ה-Frontend או מנהלי המערכת. מנהל את קבלת הבקשות, אימות ההרשאות, והעברת הלוגיקה לשכבת ה-Services.

פונקציות ושיטות במחלקה: `metalsCandles()` , `forexCandles()` , `stockCandles()` , `livePrice()`

תיאור ודוקומנטציה: REST controller providing market data endpoints for the frontend. Delivers real-time quotes and historical candlestick series for charting.

```

package com.brokerui.market;

import com.brokerui.broker.MT5IntegrationService;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;
import java.util.Map;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

/**
 * REST controller providing market data endpoints for the frontend.
 * Delivers real-time quotes and historical candlestick series for charting.
 */
@RestController
public class MarketController {

    private final MT5IntegrationService mt5;
    private final YahooFinanceService yahoo;
    private final BinanceService binance;
    private final MarketPriceService priceService;

    /**
     * Constructs the MarketController with integrated market services.
     *
     * @param mt5 MetaTrader 5 service integration
     * @param yahoo Yahoo Finance service integration
     * @param binance Binance crypto price service integration
     * @param priceService general market price aggregator
     */
    public MarketController(MT5IntegrationService mt5, YahooFinanceService yahoo, BinanceService binance, MarketPriceService priceService) {
        this.mt5 = mt5;
        this.yahoo = yahoo;
        this.binance = binance;
        this.priceService = priceService;
    }

    /**
     * Retrieves the current real-time live price quote for the symbol.
     *
     * @param symbol symbol code (e.g. EURUSD)
     * @return response containing current price, symbol, and source
     */
    @GetMapping("/api/market/price/{symbol}")
    public ResponseEntity<> livePrice(@PathVariable String symbol) {
        String sym = symbol == null ? "" : symbol.trim().toUpperCase();
        double p = priceService.getLivePrice(sym);
    }

```


פרק 7: מילון מפורט - 15) Core Services & Integrations

שירות עסקי / ליבה

com.brokerui.broker.MT5BridgeService

נתיב קובץ: MT5_Infrastructure_v2/Backend/MT5BridgeService.java | סוג: class | אנוטציות: @Value, @Service | שורות קוד: 120

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `executeTrade()`, `isMT5Symbol()`, `getCandles()`

תיאור ודוקומנטציה: STANDALONE MT5 BRIDGE INFRASTRUCTURE This service handles the communication with MT5 for: 1. Chart Data (Candles) 2. Real-time Price 3. Order Execution

```

package com.brokerui.broker;

import org.json.JSONArray;
import org.json.JSONObject;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.nio.file.Files;
import java.nio.file.StandardOpenOption;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Locale;
import java.util.concurrent.TimeUnit;

/**
 * STANDALONE MT5 BRIDGE INFRASTRUCTURE
 * This service handles the communication with MT5 for:
 * 1. Chart Data (Candles)
 * 2. Real-time Price
 * 3. Order Execution
 */
@Service
public class MT5BridgeService {

    @Value("${broker.mt5.base-path}")
    private String basePath; // Path to MT5/MQL5/Files directory

    private static final String MT5_HOST = "127.0.0.1";
    private static final int MT5_PORT = 5555;
    private static final DateTimeFormatter MT5_DATE_FMT = DateTimeFormatter.ofPattern("yyyy.MM.dd HH:mm:ss");

    /**
     * Executes a trade on MT5 via Socket.
     */
    public String executeTrade(String symbol, String action, double lot, double tp, double sl) {
        try (Socket socket = new Socket()) {
            socket.connect(new InetSocketAddress(MT5_HOST, MT5_PORT), 2000);
            try (PrintWriter writer = new PrintWriter(socket.getOutputStream(), true)) {
                // Command Format: TRADE|SYMBOL|ACTION|PRICE|TP|SL|LOT
                String command = String.format(Locale.US, "TRADE%s|s|0|%.5f|%.5f|%.2f",
                    symbol.trim().toUpperCase(), action.toUpperCase(), tp, sl, lot);
                writer.println(command);
            }
        }
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/auth/AppUserDetailsService.java | סוג: class
| אנוטציות: @param, @throws, @return, @Override, @Service | שורות קוד: 57

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `loadUserByUsername()`

תיאור ודוקומנטציה: Custom UserDetailsService implementation. Integrates AppUser entities from the database with Spring Security.

```

package com.brokerui.auth;

import com.brokerui.user.AppUser;
import com.brokerui.user.AppUserRepository;
import java.util.List;
import java.util.Locale;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.authentication.DisabledException;
import org.springframework.stereotype.Service;

/**
 * Custom UserDetailsService implementation.
 * Integrates AppUser entities from the database with Spring Security.
 */
@Service
public class AppUserDetailsService implements UserDetailsService {
    private final AppUserRepository repo;

    /**
     * Constructs the AppUserDetailsService.
     *
     * @param repo the user accounts database repository
     */
    public AppUserDetailsService(AppUserRepository repo) {
        this.repo = repo;
    }

    /**
     * Loads user authentication details from the database by email address.
     * Checks if user is banned or missing a password.
     *
     * @param email user email address
     * @return Spring Security UserDetails representation of the authenticated user
     * @throws UsernameNotFoundException if user or password hash is not found
     * @throws DisabledException if the user has been banned
     */
    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        AppUser u =
            repo.findByEmailIgnoreCase(email.toLowerCase(Locale.ROOT))
                .orElseThrow(() -> new UsernameNotFoundException(email));
        if (u.getPasswordHash() == null || u.getPasswordHash().isBlank()) {
            throw new UsernameNotFoundException("no password");
        }
        if (u.isBanned()) {
            throw new DisabledException("banned");
        }
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/AlpacaClient.java | סוג: class |
אנוטציות: @return, @param, @throws | שורות קוד: 250

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: שירות עזר ללא מתודות ציבוריות נוספות

תיאור ודוקומנטציה: Client class that acts as a file-bridge connector to the MetaTrader 5 (MT5) client terminal. Reads and writes request files to prompt the MT5 Expert Advisor (EA) for current prices and .historical/recent candlestick indicators

```

package com.brokerui.broker;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

import java.nio.file.Files;
import java.nio.file.Paths;
import java.nio.file.StandardOpenOption;
import java.time.Duration;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.util.concurrent.TimeUnit;

import org.json.*;

/**
 * Client class that acts as a file-bridge connector to the MetaTrader 5 (MT5) client terminal.
 * Reads and writes request files to prompt the MT5 Expert Advisor (EA)
 * for current prices and historical/recent candlestick indicators.
 */
public class AlpacaClient {

    private static final String REQUEST_FILE = "C:\\\\Users\\david\\AppData\\Roaming\\MetaQuote";
    private static final String CANDLE_FILE = "C:\\\\Users\\david\\AppData\\Roaming\\MetaQuote";
    private static final String REQUEST_PRICE_FILE = "C:\\\\Users\\david\\AppData\\Roaming\\Me";
    private static final String PRICE_FILE = "C:\\\\Users\\david\\AppData\\Roaming\\MetaQuote";

    /**
     * Requests recent candlestick bars from the MT5 EA terminal.
     *
     * @param symbol trading symbol (e.g. EURUSD)
     * @param count count of candles to parse
     * @param timeframe timeframe period identifier (e.g. 1Hour)
     * @return array of populated Candles (candles[0] being the newest)
     * @throws Exception if connection times out or file access fails
     */
    public static Candle[] getRecentCandles(String symbol, int count, String timeframe) thro
        File candleFile = new File(CANDLE_FILE);
        File requestFile = new File(REQUEST_FILE);

        // 🚀 ניקוי קובץ גרסה אם קיים
        if (candleFile.exists() && candleFile.length() > 5) {
            System.out.println("⚠️ candles_data.json מרוקן תוכן - לא ריק...");
            new FileWriter(candleFile, false).close();
        }
    }

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/AuditLogService.java | סוג: class | שורות קוד: 77
אנוטציות: @return, @param, @Service

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `log()`

תיאור ודוקומנטציה: Centralized audit logging service. Writes every critical user action to the audit_log table. Never throws – audit failures must not interrupt business logic.

```

package com.brokerui.broker;

import com.brokerui.user.AppUser;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.stereotype.Service;

/**
 * Centralized audit logging service.
 * Writes every critical user action to the audit_log table.
 * Never throws - audit failures must not interrupt business logic.
 */
@Service
public class AuditLogService {

    private final AuditLogRepository auditLogRepo;

    /**
     * Constructs the AuditLogService with AuditLogRepository.
     *
     * @param auditLogRepo the repository for audit logs
     */
    public AuditLogService(AuditLogRepository auditLogRepo) {
        this.auditLogRepo = auditLogRepo;
    }

    /**
     * Records a user activity in the audit log database.
     * The goal of this method is to safely write trace records without throwing exceptions.
     *
     * @param user the acting user (may be null for failed logins)
     * @param action short action code, e.g. "LOGIN_SUCCESS", "ORDER_PLACED"
     * @param detail human-readable detail string
     * @param req current HTTP request for IP / User-Agent extraction (may be null)
     */
    public void log(AppUser user, String action, String detail, HttpServletRequest req) {
        try {
            AuditLog entry = new AuditLog();
            entry.setUser(user);
            entry.setAction(action);
            entry.setDetail(detail);
            if (req != null) {
                entry.setIp(extractIp(req));
                entry.setUserAgent(truncate(req.getHeader("User-Agent"), 512));
            }
            auditLogRepo.save(entry);
        } catch (Exception ex) {
            // Log to stderr but never propagate - audit must not break the request
            System.err.println("[AuditLog] save failed: " + ex.getMessage());
        }
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/MT5IntegrationService.java | סוג: class | אנוטציות: @param, @throws, @Value, @return, @Service | שורות קוד: 249

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `isConfigured()`, `sendTrade()`, `getOpenPositions()`, `getRecentCandles()`, `getPrice()`, `checkHealth()`, `getBasePath()`

תיאור ודוקומנטציה: Orchestrates the MetaTrader 5 bridge: 1. File Bridge (getting candles and prices) - based on AlpacaClient logic. 2. Socket Bridge (executing trades) - based on MT5JavaTradeWriter logic.

```

package com.brokerui.broker;

import org.json.JSONArray;
import org.json.JSONObject;
import java.util.Locale;
import java.util.List;
import java.util.ArrayList;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.nio.file.Files;
import java.nio.file.StandardOpenOption;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.util.concurrent.TimeUnit;

/**
 * Orchestrates the MetaTrader 5 bridge:
 * 1. File Bridge (getting candles and prices) - based on AlpacaClient logic.
 * 2. Socket Bridge (executing trades) - based on MT5JavaTradeWriter logic.
 */
@Service
public class MT5IntegrationService {

    @Value("${broker.mt5.base-path}")
    private String basePath;

    private static final String MT5_HOST = "127.0.0.1";
    private static final int MT5_PORT = 5555;

    private final DateTimeFormatter fileFmt = DateTimeFormatter.ofPattern("yyyy.MM.dd HH:mm");

    /**
     * Checks if the MetaTrader 5 base path is configured.
     *
     * @return true if configured, false otherwise
     */
    public boolean isConfigured() {
        return basePath != null && !basePath.isBlank();
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/MT5PositionSyncService.java | סוג: class | אנוטציות: @Scheduled, @param, @Transactional, @Service | שורות קוד: 106

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `sync()`

תיאור ודוקומנטציה: Scheduled service that periodically syncs MT5 terminal open positions into the local JPA positions repository.

```

package com.brokerui.broker;

import org.json.JSONArray;
import org.json.JSONObject;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;

/**
 * Scheduled service that periodically syncs MT5 terminal open positions
 * into the local JPA positions repository.
 */
@Service
public class MT5PositionSyncService {

    private final MT5IntegrationService mt5Service;
    private final PositionRepository positionRepo;
    private final TradingAccountRepository accountRepo;

    /**
     * Constructs the MT5PositionSyncService.
     *
     * @param mt5Service MetaTrader 5 service integration
     * @param positionRepo database position repository
     * @param accountRepo database trading account repository
     */
    public MT5PositionSyncService(MT5IntegrationService mt5Service, PositionRepository positionRepo,
                                   TradingAccountRepository accountRepo) {
        this.mt5Service = mt5Service;
        this.positionRepo = positionRepo;
        this.accountRepo = accountRepo;
    }

    /**
     * Scheduled task running every 10 seconds to sync open positions from MT5.
     * The goal is to keep the local DB in alignment with the MT5 Expert Advisor's active trades.
     */
    @Scheduled(fixedDelay = 10000) // Every 10 seconds
    @Transactional
    public void sync() {
        if (!mt5Service.isConfigured()) return;
        try {
            JSONArray mt5Positions = mt5Service.getOpenPositions();
            if (mt5Positions == null) return;

            // For the purpose of this project, we sync the first active account (Demo Account)
            Optional<TradingAccount> taOpt = accountRepo.findAll().stream().findFirst();

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/MarginLoanService.java | סוג: class |
 אנוטציות: @Lazy, @param, @Transactional, @Scheduled, @code, @Autowired, @return, @link, @Service
 שורות קוד: 416

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `accrueDueInterest()`, `checkLiquidationsTick()`, `repaySettlementOrBorrow()`,
`checkLiquidations()`, `accrueDueInterestTick()`, `liquidateAccount()`, `tryCoverShortfall()`,
`computeMarginLevel()`, `accrueDailyInterest()`

תיאור ודוקומנטציה: Implements a Bybit-style margin credit line: when a trade needs more cash than the account currently holds, the shortfall is automatically borrowed against a fixed per-account credit limit. Borrowed funds accrue daily interest and, if the account's equity falls too close to the outstanding debt, open positions are force-liquidated to protect

```

package com.brokerui.broker;

import com.brokerui.market.MarketPriceService;
import com.brokerui.user.AppUser;
import jakarta.servlet.http.HttpServletRequest;
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Instant;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Lazy;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

/**
 * Implements a Bybit-style margin credit line: when a trade needs more cash than
 * the account currently holds, the shortfall is automatically borrowed against a
 * fixed per-account credit limit. Borrowed funds accrue daily interest and, if the
 * account's equity falls too close to the outstanding debt, open positions are
 * force-liquidated to protect the broker from bad debt.
 */
@Service
public class MarginLoanService {

    /** Fixed credit line available to every demo account, regardless of equity. */
    public static final BigDecimal CREDIT_LIMIT = new BigDecimal("10000");

    /** Daily interest rate charged on the outstanding borrowed balance. */
    public static final BigDecimal DAILY_INTEREST_RATE = new BigDecimal("0.005"); // 0.5%/day

    /** Below this margin level (equity / debt) a warning ("margin call") is shown. */
    public static final BigDecimal MARGIN_CALL_LEVEL = new BigDecimal("1.10"); // 110%

    /** Below this margin level positions are force-liquidated. */
    public static final BigDecimal LIQUIDATION_LEVEL = new BigDecimal("1.00"); // 100%

    private final TradingAccountRepository accountRepo;
    private final PositionRepository positionRepo;
    private final BrokerOrderRepository orderRepo;
    private final MarginLoanLedgerRepository ledgerRepo;
    private final NotificationRepository notificationRepo;
    private final MarketPriceService priceService;
    private final AuditLogService auditLogService;

    @Autowired @Lazy
    private MarginLoanService self;

    public MarginLoanService(
        TradingAccountRepository accountRepo,

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/NNPredictorClient.java | סוג: class |
אנוטציות: @return, @param, @Service | שורות קוד: 57

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: שירות עזר ללא מתודות ציבוריות נוספות

תיאור ודוקומנטציה: Client service connecting to the external Neural Network python server (port 3005). Sends feature arrays to get execution routing recommendations

```

package com.brokerui.broker;

import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;
import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import java.util.HashMap;
import java.util.Map;
import java.util.List;

/**
 * Client service connecting to the external Neural Network python server (port 3005).
 * Sends feature arrays to get execution routing recommendations.
 */
@Service
public class NNPredictorClient {
    private final RestTemplate restTemplate = new RestTemplate();
    private final String predictUrl = "http://localhost:3005/predict";

    /**
     * Posts a feature array to the NN model server and returns prediction metrics.
     * The goal is to obtain routing recommendation labels (e.g. INTERNAL, EXTERNAL).
     *
     * @param features array of double values representing model inputs
     * @return map containing matchProb, expectedSavings, and routeRecommendation, or null if query
     */
    public Map<String, Object> getPrediction(double[] features) {
        try {
            HttpHeaders headers = new HttpHeaders();
            headers.setContentType(MediaType.APPLICATION_JSON);

            Map<String, Object> requestBody = new HashMap<>();
            requestBody.put("features", features);

            HttpEntity<Map<String, Object>> entity = new HttpEntity<>(requestBody, headers);
            Map response = restTemplate.postForObject(predictUrl, entity, Map.class);

            if (response != null && response.containsKey("prediction")) {
                Object pred = response.get("prediction");
                if (pred instanceof List) {
                    List<?> list = (List<?>) pred;
                    if (list.size() >= 3) {
                        Map<String, Object> result = new HashMap<>();
                        result.put("matchProb", list.get(0));
                        result.put("expectedSavings", list.get(1));
                        result.put("routeRecommendation", list.get(2));
                        return result;
                    }
                }
            }
        }
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/OrderExecutionService.java | סוג: class | אנוטציות: @Lazy, @param, @Transactional, @Scheduled, @Autowired, @return, @Service | שורות קוד: 517

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `tick()`, `closePositionDueToSlTp()`, `tryExecute()`

תיאור ודוקומנטציה: Core service class for executing and managing limit, stop, and market orders. Feeds live pricing quotes, executes orders when conditions match, recalculates accounts equity, calls the Python NN model advisor, and routes orders either internally or forwards them to the MT5 bridge.

```

package com.brokerui.broker;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Instant;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Lazy;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

/**
 * Core service class for executing and managing limit, stop, and market orders.
 * Feeds live pricing quotes, executes orders when conditions match, recalculates accounts equity,
 * calls the Python NN model advisor, and routes orders either internally or forwards them to the MT
 */
@Service
public class OrderExecutionService {
    private final TradingAccountRepository accountRepo;
    private final PositionRepository positionRepo;
    private final BrokerOrderRepository orderRepo;
    private final TradeFillRepository fillRepo;
    private final NotificationRepository notificationRepo;
    private final SymbolRepository symbolRepo;
    private final MT5IntegrationService mt5Service;
    private final com.brokerui.market.MarketPriceService priceService;
    private final NNPredictorClient nnPredictorClient;
    private final MarginLoanService marginLoanService;
    private final AccountTransactionRepository txRepo;

    @Autowired @Lazy
    private OrderExecutionService self;

    /**
     * Constructs the OrderExecutionService with required components.
     *
     * @param accountRepo database trading account repository
     * @param positionRepo database position repository
     * @param orderRepo database order repository
     * @param fillRepo database trade fill repository
     * @param notificationRepo database notification alerts repository
     * @param symbolRepo database symbols configuration repository
     * @param mt5Service MetaTrader 5 service integration
     * @param priceService general market price aggregator
     * @param nnPredictorClient Neural Network server client adapter
     * @param marginLoanService margin credit line service
     */
    public OrderExecutionService(
        TradingAccountRepository accountRepo,

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/TradingFees.java | סוג: class | אנוטציות: @code, @param, @return | שורות קוד: 108

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: שירות עזר ללא מתודות ציבוריות נוספות

תיאור ודוקומנטציה: Central place for trading fee constants charged to the client on every trade fill. This is intentionally the same flat amount used by the netting-broker AI routing model (see `buysellmodel/simulation.js`, `EXCHANGE_FEE_PER_TRADE`) as the assumed cost of routing an order to the external exchange/liquidity provider. That

```

package com.brokerui.broker;

import java.math.BigDecimal;

/**
 * Central place for trading fee constants charged to the client on every trade fill.
 * <p>
 * This is intentionally the same flat amount used by the netting-broker AI routing model
 * (see {@code buysellmodel/simulation.js}, {@code EXCHANGE_FEE_PER_TRADE}) as the assumed
 * cost of routing an order to the external exchange/liquidity provider. That symmetry keeps
 * the economics consistent: the client always pays this commission regardless of how the
 * order was routed, while the platform only incurs the matching exchange-fee cost when the
 * AI advisor ({@code NNPredictorClient}) recommends {@code EXTERNAL} routing. Orders matched
 * {@code INTERNAL}ly cost the platform nothing, so the whole commission becomes pure profit –
 * which is exactly the netting-broker business model this app demonstrates.
 */
public final class TradingFees {
    private TradingFees() {}

    /** Default fallback commission charged when trade parameters are not fully specified. */
    public static final BigDecimal COMMISSION_PER_TRADE = new BigDecimal("1.50");

    /**
     * Dynamically calculates commission for a trade based on notional value, asset class tier, and va
     * <p>
     * Formula:
     * Base Fee ($1.00) + (Price * Quantity * ContractSize * 0.0002 * AssetMultiplier)
     * Subject to minimum $1.00 and maximum $25.00 cap to ensure high broker profitability while prese
     *
     * @param symbolCode the asset symbol code
     * @param quantity trade volume / quantity
     * @param fillPrice execution price
     * @return calculated commission amount
     */
    public static BigDecimal calculateCommission(String symbolCode, BigDecimal quantity, BigDecimal fi
        if (quantity == null || quantity.compareTo(BigDecimal.ZERO) <= 0 || fillPrice == null || fillPri
            return COMMISSION_PER_TRADE;
        }

        BigDecimal contractSize = BrokerApiController.getContractSize(symbolCode);
        BigDecimal notional = fillPrice.multiply(quantity).multiply(contractSize);

        // Fixed commission rate of 0.0025% (2.5 bps) of notional value
        BigDecimal rate = new BigDecimal("0.000025");
        BigDecimal totalFee = notional.multiply(rate).setScale(2, java.math.RoundingMode.HALF_UP);

        // Minimum $0.10, Maximum $50.00 per trade to stay logical and avoid eating all profit or losing
        BigDecimal minFee = new BigDecimal("0.10");
        BigDecimal maxFee = new BigDecimal("50.00");

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/market/BinancePriceService.java | סוג: class
| אנוטציות: @return, @param, @throws, @Service | שורות קוד: 109

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `getLastPrice()`

תיאור ודוקומנטציה: Service for fetching live market prices from the Binance REST API

```

package com.brokerui.market;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.math.BigDecimal;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.RestClientException;
import org.springframework.http.client.SimpleClientHttpRequestFactory;

/**
 * Service for fetching live market prices from the Binance REST API.
 */
@Service
public class BinancePriceService {
    private static final String BASE = "https://api.binance.com";

    private final RestClient http;
    private final ObjectMapper objectMapper;

    /**
     * Constructs the BinancePriceService with HTTP rest client configurations.
     *
     * @param objectMapper Jackson JSON object mapper
     */
    public BinancePriceService(ObjectMapper objectMapper) {
        this.objectMapper = objectMapper;
        SimpleClientHttpRequestFactory rf = new SimpleClientHttpRequestFactory();
        rf.setConnectTimeout(3000);
        rf.setReadTimeout(4000);
        this.http =
            RestClient.builder()
                .baseUrl(BASE)
                .requestFactory(rf)
                .build();
    }

    /**
     * Fetches the last traded price for a specific symbol from the Binance API.
     * The goal of this method is to fetch the price, parsing the ticker response to return it.
     *
     * @param symbolCode the internal system symbol code (e.g. BTCUSD, EURUSD)
     * @return the last price as a BigDecimal
     * @throws IllegalArgumentException if the symbol is empty/invalid
     * @throws IllegalStateException if the price cannot be fetched
     */
    public BigDecimal getLastPrice(String symbolCode) {
        String rawSym = symbolCode == null ? "" : symbolCode.trim().toUpperCase();
        String sym = mapToBinanceSymbol(rawSym);
        if (sym.isEmpty()) {

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/market/BinanceService.java | סוג: class |
אנוטציות: @return, @param, @throws, @Service | שורות קוד: 159

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `isConfigured()`, `getLivePrice()`, `getCandles()`

תיאור ודוקומנטציה: Service to interact with the Bybit REST API for retrieving market tickers and historical klines/candles.

```

package com.brokerui.market;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.stereotype.Service;

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Service to interact with the Bybit REST API for retrieving market tickers and historical klines/c
 */
@Service
public class BinanceService {

    private static final String BYBIT_PRICE_URL = "https://api.bybit.com/v5/market/tickers?category=";
    private static final String BYBIT_KLINES_URL = "https://api.bybit.com/v5/market/kline?category=";

    private final HttpClient httpClient;
    private final ObjectMapper om;

    private final Map<String, double[]> priceCache = new ConcurrentHashMap<>();
    private static final long CACHE_TTL_MS = 2000; // 2 seconds for live price

    /**
     * Constructs the BinanceService with customized ObjectMapper.
     *
     * @param om Jackson JSON object mapper
     */
    public BinanceService(ObjectMapper om) {
        this.om = om;
        this.httpClient = HttpClient.newBuilder()
            .followRedirects(HttpClient.Redirect.ALWAYS)
            .build();
    }

    /**
     * Checks if this market service configuration is complete.
     *
     * @return true, as Bybit API endpoints require no custom API key configurations
     */
    public boolean isConfigured() {
        return true;
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/market/FinnhubMarketService.java | סוג: class
| אנוטציות: @param, @throws, @Value, @return, @Service | שורות קוד: 247

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `isConfigured()`, `metalsCandles()`, `oandaQuote()`, `usStockCandles()`, `forexCandles()`

תיאור ודוקומנטציה: Service to retrieve market data, quotes, and klines/candles from the Finnhub REST API. Primarily handles US stock, forex, and precious metals via OANDA integration.

```

package com.brokerui.market;

import com.fasterxml.jackson.core.JsonProcessingException;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;

/**
 * Service to retrieve market data, quotes, and klines/candles from the Finnhub REST API.
 * Primarily handles US stock, forex, and precious metals via OANDA integration.
 */
@Service
public class FinnhubMarketService {
    private static final String BASE = "https://finnhub.io/api/v1";

    private final RestClient http;
    private final ObjectMapper objectMapper;
    private final String apiKey;

    /**
     * Constructs the FinnhubMarketService with ObjectMapper and configured API key.
     *
     * @param objectMapper Jackson JSON object mapper
     * @param apiKey the Finnhub API token
     */
    public FinnhubMarketService(
        ObjectMapper objectMapper,
        @Value("${broker.market.finnhub-api-key:}") String apiKey) {
        this.objectMapper = objectMapper;
        this.apiKey = apiKey == null ? "" : apiKey.trim();
        this.http = RestClient.builder().baseUrl(BASE).build();
    }

    /**
     * Checks if the Finnhub API integration is active and has a configured API key.
     *
     * @return true if an API key is configured, false otherwise
     */
    public boolean isConfigured() {
        return !apiKey.isEmpty();
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/market/MarketPriceService.java | סוג: class |
אנוטציות: @return, @param, @Service | שורות קוד: 112

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `getLivePrice()`

תיאור ודוקומנטציה: Aggregator service that coordinates fetching live prices from multiple adapters
(Binance for crypto, MT5 for FX/metals, Yahoo Finance as secondary, and synthetic fallback)

```

package com.brokerui.market;

import com.brokerui.broker.MT5IntegrationService;
import java.math.BigDecimal;
import java.math.RoundingMode;
import org.springframework.stereotype.Service;

/**
 * Aggregator service that coordinates fetching live prices from multiple adapters
 * (Binance for crypto, MT5 for FX/metals, Yahoo Finance as secondary, and synthetic fallback).
 */
@Service
public class MarketPriceService {
    private final MT5IntegrationService mt5;
    private final YahooFinanceService yahoo;
    private final BinanceService binance;

    /**
     * Constructs the MarketPriceService with required data adapters.
     *
     * @param mt5 MetaTrader 5 service integration
     * @param yahoo Yahoo Finance service integration
     * @param binance Binance crypto price service integration
     */
    public MarketPriceService(MT5IntegrationService mt5, YahooFinanceService yahoo, BinanceService binance) {
        this.mt5 = mt5;
        this.yahoo = yahoo;
        this.binance = binance;
    }

    /**
     * Attempts to query live price quote from all configured feeds in sequence.
     * The goal is to return a valid non-zero quote.
     *
     * @param symbol symbol code (e.g. BTCUSD, EURUSD)
     * @return live quote as double
     */
    public double getLivePrice(String symbol) {
        String sym = symbol == null ? "" : symbol.trim().toUpperCase();

        // 0. Try Binance for crypto
        if (isCrypto(sym)) {
            try {
                double p = binance.getLivePrice(sym);
                if (p > 0) return p;
            } catch (Exception ignored) {}
        }

        // 1. Try MT5 if configured
        if (mt5.isConfigured()) {

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/market/YahooFinanceService.java | סוג: class
| אנוטציות: @return, @param, @throws, @Service | שורות קוד: 219

תפקיד המחלקה במערכת: שירות עסקי מרכזי (Service / Client). מנהל את הלוגיקה המורכבת של המסחר, חישובי סיכונים, אשראי, ניתוב AI, או הזרמת מחירים.

פונקציות ושיטות במחלקה: `getLivePrice()`, `getCandles()`, `isConfigured()`, `checkClientTrusted()`, `getAcceptedIssuers()`, `checkServerTrusted()`

תיאור ודוקומנטציה: Fetches Forex/Metals live prices and OHLCV candles from Yahoo Finance public v7 API.

```

package com.brokerui.market;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import org.springframework.stereotype.Service;

import javax.net.ssl.SSLContext;
import javax.net.ssl.TrustManager;
import javax.net.ssl.X509TrustManager;
import java.net.URI;
import java.net.URLEncoder;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.nio.charset.StandardCharsets;
import java.security.SecureRandom;
import java.security.cert.X509Certificate;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Fetches Forex/Metals live prices and OHLCV candles from Yahoo Finance public v7 API.
 */
@Service
public class YahooFinanceService {

    private static final String YAHOO_CHART_URL = "https://query1.finance.yahoo.com/v7/finance/chart";
    private static final String UA = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/80.0.3987.163 Safari/537.36";

    private final Map<String, double[]> priceCache = new ConcurrentHashMap<>();
    private static final long CACHE_TTL_MS = 5_000; // 5-second TTL price cache

    private final HttpClient httpClient;
    private final ObjectMapper om;

    /**
     * Constructs the YahooFinanceService.
     *
     * @param om Jackson JSON object mapper
     */
    public YahooFinanceService(ObjectMapper om) {
        this.om = om;
        this.httpClient = buildHttpClient();
    }

    private static HttpClient buildHttpClient() {
        try {
            TrustManager[] trustAll = { new X509TrustManager() {

```

פרק 8: מילון מפורט - 6) Configs, Security & Setup (מחלקות)

תצורה ואבטחה

com.brokerui.BackendApplication

נתיב קובץ: backend/backend/src/main/java/com/brokerui/BackendApplication.java | סוג: class | אנוטציות: @param, @SpringBootApplication, @EnableScheduling | שורות קוד: 22

תפקיד המחלקה במערכת: הגדרת קונפיגורציית Spring Boot, אבטחת Spring Security, הגנות CSRF/CORS, או אתחול נתונים בהפעלת השרת.

פונקציות ושיטות במחלקה: קונפיגורציית Spring Beans / Bootstrapping

```
package com.brokerui;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

/**
 * The main application launcher class for the Spring Boot application.
 */
@SpringBootApplication
@EnableScheduling
public class BackendApplication {
    /**
     * The entry point of the Spring Boot JVM application.
     *
     * @param args command line arguments
     */
    public static void main(String[] args) {
        SpringApplication.run(BackendApplication.class, args);
    }
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/auth/AppUserDetailsService.java | סוג: class
| אנוטציות: @param, @throws, @return, @Override, @Service | שורות קוד: 57

תפקיד המחלקה במערכת: הגדרת קונפיגורציית Spring Boot, אבטחת Spring Security, הגנות CSRF/CORS, או אתחול נתונים בהפעלת השרת.

פונקציות ושיטות במחלקה: `loadUserByUsername()`

```

package com.brokerui.auth;

import com.brokerui.user.AppUser;
import com.brokerui.user.AppUserRepository;
import java.util.List;
import java.util.Locale;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.security.authentication.DisabledException;
import org.springframework.stereotype.Service;

/**
 * Custom UserDetailsService implementation.
 * Integrates AppUser entities from the database with Spring Security.
 */
@Service
public class AppUserDetailsService implements UserDetailsService {
    private final AppUserRepository repo;

    /**
     * Constructs the AppUserDetailsService.
     *
     * @param repo the user accounts database repository
     */
    public AppUserDetailsService(AppUserRepository repo) {
        this.repo = repo;
    }

    /**
     * Loads user authentication details from the database by email address.
     * Checks if user is banned or missing a password.
     *
     * @param email user email address
     * @return Spring Security UserDetails representation of the authenticated user
     * @throws UsernameNotFoundException if user or password hash is not found
     * @throws DisabledException if the user has been banned
     */
    @Override
    public UserDetails loadUserByUsername(String email) throws UsernameNotFoundException {
        AppUser u =
            repo.findByEmailIgnoreCase(email.toLowerCase(Locale.ROOT))
                .orElseThrow(() -> new UsernameNotFoundException(email));
        if (u.getPasswordHash() == null || u.getPasswordHash().isBlank()) {
            throw new UsernameNotFoundException("no password");
        }
        if (u.isBanned()) {
            throw new DisabledException("banned");
        }
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/auth/AuthBootstrap.java | סוג: class | אנוטציות: @param, @Component, @broker, @gmail, @Override | שורות קוד: 276

תפקיד המחלקה במערכת: הגדרת קונפיגורציית Spring Boot, אבטחת Spring Security, הגנות CSRF/CORS, או אתחול נתונים בהפעלת השרת.

פונקציות ושיטות במחלקה: run()

```

package com.brokerui.auth;

import com.brokerui.broker.MarginLoanLedger;
import com.brokerui.broker.MarginLoanLedgerRepository;
import com.brokerui.broker.Position;
import com.brokerui.broker.PositionRepository;
import com.brokerui.broker.TradingAccount;
import com.brokerui.broker.TradingAccountRepository;
import com.brokerui.user.AppUser;
import com.brokerui.user.AppUserRepository;
import com.brokerui.user.UserRole;
import java.math.BigDecimal;
import java.time.Instant;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Component;

/**
 * Application startup runner that bootstraps demo database records.
 * Creates default demo users across 4 distinct use-case scenarios.
 */
@Component
public class AuthBootstrap implements ApplicationRunner {
    private final AppUserRepository repo;
    private final PasswordEncoder encoder;
    private final TradingAccountRepository accountRepo;
    private final MarginLoanLedgerRepository ledgerRepo;
    private final PositionRepository positionRepo;

    /**
     * Constructs the AuthBootstrap runner.
     *
     * @param repo user repository
     * @param encoder password encoder utility
     * @param accountRepo trading account repository
     * @param ledgerRepo margin loan ledger repository
     * @param positionRepo position repository
     */
    public AuthBootstrap(
        AppUserRepository repo,
        PasswordEncoder encoder,
        TradingAccountRepository accountRepo,
        MarginLoanLedgerRepository ledgerRepo,
        PositionRepository positionRepo) {
        this.repo = repo;
        this.encoder = encoder;
        this.accountRepo = accountRepo;
        this.ledgerRepo = ledgerRepo;
        this.positionRepo = positionRepo;
    }

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/config/LocalhostOnlyFilter.java | סוג: class
| אנוטציות: @param, @Component, @throws, @Order, @Override | שורות קוד: 66

תפקיד המחלקה במערכת: הגדרת קונפיגורציית Spring Boot, אבטחת Spring Security, הגנות CSRF/CORS, או אתחול נתונים בהפעלת השרת.

פונקציות ושיטות במחלקה: קונפיגורציית Spring Beans / Bootstrapping

```

package com.brokerui.config;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.Set;

/**
 * Security filter that restricts ALL /api/admin/** endpoints to localhost only.
 * Any request originating from outside 127.0.0.1 / ::1 is rejected with HTTP 403.
 *
 * This runs BEFORE Spring Security so no token, session, or role can bypass it
 * from a remote machine - the connection is dropped at the filter level.
 */
@Component
@Order(1) // Run before everything else
public class LocalhostOnlyFilter extends OncePerRequestFilter {

    private static final Set<String> LOCALHOST_ADDRESSES = Set.of(
        "127.0.0.1",
        "::1",
        "0:0:0:0:0:0:0:1" // IPv6 loopback long form
    );

    /**
     * Inspects incoming HTTP requests. If the request is for an admin resource,
     * verifies that the remote address is a localhost IP (127.0.0.1 or IPv6 loopback).
     * If not, blocks the request and returns HTTP 403 Forbidden.
     *
     * @param request the servlet request
     * @param response the servlet response
     * @param filterChain the servlet filter chain
     * @throws ServletException if a servlet exception occurs
     * @throws IOException if an I/O exception occurs
     */
    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {

        String path = request.getRequestURI();

        // Restrict both ADMIN API and the ADMIN HTML page to localhost

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/config/SecurityConfig.java | סוג: class |
אנוטציות: @Lazy, @param, @EnableWebSecurity, @throws, @Bean, @return, @Configuration | שורות קוד: 221

תפקיד המחלקה במערכת: הגדרת קונפיגורציית Spring Boot, אבטחת Spring Security, הגנות CSRF/CORS, או אתחול נתונים בהפעלת השרת.

פונקציות ושיטות במחלקה: passwordEncoder(), filterChain(), corsConfigurationSource(), daoAuthenticationProvider()

```

package com.brokerui.config;

import com.brokerui.auth.AppUserDetailsService;
import com.brokerui.broker.AuditLogService;
import com.brokerui.user.AppUserRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Lazy;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.access.expression.WebExpressionAuthorizationManager;
import org.springframework.security.web.authentication.AuthenticationFailureHandler;
import org.springframework.security.web.authentication.AuthenticationSuccessHandler;
import org.springframework.security.web.csrf.CookieCsrfTokenRepository;
import org.springframework.security.web.csrf.CsrfTokenRequestAttributeHandler;
import org.springframework.security.web.util.matcher.AntPathRequestMatcher;
import org.springframework.security.authentication.DisabledException;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;
import java.util.Arrays;

/**
 * Configuration class for Spring Security.
 * Defines password encoders, CORS policy, user auth providers, and requests routing rules.
 */
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    // @Lazy prevents circular dependency: SecurityConfig → AuditLogService → AuditLogRepository
    private final AuditLogService auditLogService;
    private final AppUserRepository appUserRepository;

    /**
     * Constructs the SecurityConfig with lazily loaded audit and user repos.
     *
     * @param auditLogService the security audit logging service
     * @param appUserRepository the user accounts database repository
     */
    public SecurityConfig(@Lazy AuditLogService auditLogService,
                         @Lazy AppUserRepository appUserRepository) {

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/config/SpaRoutingConfig.java | סוג: class |
אנוטציות: @param, @throws, @return, @Configuration, @Override | שורות קוד: 77

תפקיד המחלקה במערכת: הגדרת קונפיגורציית Spring Boot, אבטחת Spring Security, הגנות CSRF/CORS, או אתחול נתונים בהפעלת השרת.

פונקציות ושיטות במחלקה: `addResourceHandlers()`

```

package com.brokerui.config;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.web.servlet.resource.PathResourceResolver;

import java.io.IOException;

/**
 * MVC Configuration for SPA (Single Page Application) routing.
 * Ensures that requests for frontend paths resolve correctly to index.html.
 */
@Configuration
public class SpaRoutingConfig implements WebMvcConfigurer {

    /**
     * Sets up handlers to resolve static resources, falling back to index.html if resource is not found.
     * This facilitates HTML5 history API navigation routing.
     *
     * @param registry resource handlers registry helper
     */
    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry
            .addResourceHandler("/**")
            .addResourceLocations("classpath:/static/")
            .resourceChain(true)
            .addResolver(new PathResourceResolver() {
                /**
                 * Resolves requested resource location path. Fallback to static index.html.
                 *
                 * @param resourcePath the relative resource file path
                 * @param location the base resource directory location
                 * @return resolved Resource object
                 * @throws IOException if file access fails
                 */
                @Override
                protected Resource getResource(String resourcePath, Resource location) throws IOException {
                    Resource requested = location.createRelative(resourcePath);
                    // Serve existing static files directly, otherwise fall back to index.html for SPA routing
                    if (requested.exists() && requested.isReadable()) {
                        return requested;
                    }
                    return new ClassPathResource("/static/index.html");
                }
            });
    }
}

```

פרק 9: מילון מפורט - JPA Entities (24 מחלקות)

מודל נתונים / הישות

com.brokerui.auth.RegisterRequest

נתיב קובץ: backend/backend/src/main/java/com/brokerui/auth/RegisterRequest.java | סוג: record | שורות קוד: 10
אנוטציות: @Size, @Email, @NotBlank

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```
package com.brokerui.auth;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record RegisterRequest(
    @NotBlank @Email String email,
    @NotBlank @Size(max = 160) String displayName,
    @NotBlank @Size(min = 8, max = 72) String password) {}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/AccountTransaction.java | סוג: class |
אנוטציות:

| @Column, @ManyToOne, @CreationTimestamp, @GeneratedValue, @Entity, @Table, @JoinColumn, @Id
שורות קוד: 124

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.math.BigDecimal;
import java.time.Instant;
import org.hibernate.annotations.CreationTimestamp;

@Entity
@Table(name = "account_transaction")
public class AccountTransaction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "trading_account_id", nullable = false)
    private TradingAccount tradingAccount;

    @Column(name = "tx_type", nullable = false, length = 16)
    private String txType;

    @Column(nullable = false, length = 16)
    private String status = "PENDING";

    @Column(nullable = false, precision = 18, scale = 8)
    private BigDecimal amount;

    @Column(nullable = false, length = 8)
    private String currency = "USD";

    @Column(length = 32)
    private String method;

    @Column(columnDefinition = "TEXT")
    private String note;

    @CreationTimestamp
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    @Column(name = "processed_at")
    private Instant processedAt;
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/AuditLog.java | סוג: class | אנוטציות: @Column, @ManyToOne, @CreationTimestamp, @GeneratedValue, @Entity, @Table, @JoinColumn, @Id | שורות קוד: 91

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import com.brokerui.user.AppUser;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.time.Instant;
import org.hibernate.annotations.CreationTimestamp;

@Entity
@Table(name = "audit_log")
public class AuditLog {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id")
    private AppUser user;

    @Column(nullable = false, length = 64)
    private String action;

    @Column(columnDefinition = "TEXT")
    private String detail;

    @Column(length = 64)
    private String ip;

    @Column(name = "user_agent", columnDefinition = "TEXT")
    private String userAgent;

    @CreationTimestamp
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    public Long getId() {
        return id;
    }

    public AppUser getUser() {
        return user;
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/BotState.java | סוג: class | אנוטציות:
ללא אנוטציות מיוחדות | שורות קוד: 123

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import java.time.Duration;
import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;

public class BotState {
    public static boolean isBotOn = true;
    public static boolean isConfirmingTrade = false;

    public static boolean isBusy() {
        return isConfirmingTrade;
    }

    // 🌸 הגבלת זמן לסיגנלים הפוכים
    private static final Duration oppositeSignalCooldown = Duration.ofDays(1);

    private static final Map<String, LastDirectionInfo> lastDirectionMap = new HashMap<>();

    private static class LastDirectionInfo {
        String direction;
        LocalDateTime time;

        LastDirectionInfo(String direction, LocalDateTime time) {
            this.direction = direction;
            this.time = time;
        }
    }

    // 🌸 ניקוי רשומות ישנות
    private static void cleanOldDirections() {
        Iterator<Map.Entry<String, LastDirectionInfo>> it = lastDirectionMap.entrySet().iterator();
        LocalDateTime now = LocalDateTime.now();
        while (it.hasNext()) {
            Map.Entry<String, LastDirectionInfo> entry = it.next();
            if (Duration.between(entry.getValue().time, now).compareTo(oppositeSignalCooldown) > 0)
                it.remove();
        }
    }

    public static void updateLastDirection(String symbol, String direction) {

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/BrokerOrder.java | סוג: class | אנוטציות: @Column, @ManyToOne, @CreationTimestamp, @GeneratedValue, @Entity, @Table, @JoinColumn, @UpdateTimestamp, @Id | שורות קוד: 267

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.math.BigDecimal;
import java.time.Instant;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

@Entity
@Table(name = "broker_order")
public class BrokerOrder {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "trading_account_id", nullable = false)
    private TradingAccount tradingAccount;

    @Column(name = "symbol_code", nullable = false, length = 32)
    private String symbolCode;

    @Column(nullable = false, length = 8)
    private String side;

    @Column(name = "order_type", nullable = false, length = 16)
    private String orderType;

    @Column(nullable = false, length = 24)
    private String status = "NEW";

    @Column(nullable = false, precision = 18, scale = 8)
    private BigDecimal quantity;

    @Column(name = "limit_price", precision = 18, scale = 8)
    private BigDecimal limitPrice;

    @Column(name = "stop_price", precision = 18, scale = 8)
    private BigDecimal stopPrice;

    @Column(name = "take_profit", precision = 18, scale = 8)
    private BigDecimal takeProfit;

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/BrokerOverviewDto.java | סוג: record | שורות קוד: 21
אנוטציות: ללא אנוטציות מיוחדות |

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```
package com.brokerui.broker;

import java.math.BigDecimal;

public record BrokerOverviewDto(
    long tradingAccountId,
    String accountType,
    String currency,
    int leverage,
    BigDecimal balance,
    BigDecimal equity,
    BigDecimal marginUsed,
    BigDecimal freeMargin,
    BigDecimal borrowedBalance,
    BigDecimal creditLimit,
    BigDecimal marginLevelPct,
    BigDecimal interestAccruedTotal,
    BigDecimal commissionPaidTotal,
    BigDecimal dailyInterestRate) {}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/Candle.java | סוג: class | אנוטציות: @Override | שורות קוד: 22

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```
package com.brokerui.broker;

public class Candle {
    public final double open, high, low, close;
    public final long timestamp;

    public Candle(double open, double high, double low, double close, long timestamp) {
        this.open = open;
        this.high = high;
        this.low = low;
        this.close = close;
        this.timestamp = timestamp;
    }

    @Override
    public String toString() {
        return String.format("Candle[open=%.5f, high=%.5f, low=%.5f, close=%.5f, time=%d]",
            open, high, low, close, timestamp);
    }
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/ClosedTradeDto.java | סוג: record |
אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 15

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```
package com.brokerui.broker;

import java.math.BigDecimal;
import java.time.Instant;

public record ClosedTradeDto(
    Long id,
    String symbolCode,
    String side,
    String orderType,
    BigDecimal quantity,
    BigDecimal entryPrice,
    BigDecimal realizedPnl,
    Instant openedAt,
    Instant closedAt) {}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/KycCase.java | סוג: class | אנוטציות: @Column, @ManyToOne, @GeneratedValue, @Entity, @Table, @JoinColumn, @Id | שורות קוד: 82

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import com.brokerui.user.AppUser;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.time.Instant;

@Entity
@Table(name = "kyc_case")
public class KycCase {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false, unique = true)
    private AppUser user;

    @Column(nullable = false, length = 16)
    private String status = "NOT_STARTED";

    @Column(name = "submitted_at")
    private Instant submittedAt;

    @Column(name = "reviewed_at")
    private Instant reviewedAt;

    @Column(columnDefinition = "TEXT")
    private String note;

    public Long getId() {
        return id;
    }

    public AppUser getUser() {
        return user;
    }

    public void setUser(AppUser user) {
        this.user = user;
    }

    public String getStatus() {

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/MT5ConnectionManager.java | סוג: class
| אנוטציות: @return, @param, @Value, @Service | שורות קוד: 72

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

/**
 * Service that manages active/inactive status flag of the MT5 bridge connection.
 * Supports administration connection control.
 */
@Service
public class MT5ConnectionManager {

    private final MT5IntegrationService mt5Service;
    private boolean isConnected;

    /**
     * Constructs the MT5ConnectionManager.
     *
     * @param mt5Service MetaTrader 5 service integration
     * @param autoConnect initial connection status flag
     */
    public MT5ConnectionManager(
        MT5IntegrationService mt5Service,
        @Value("${broker.mt5.auto-connect:false}") boolean autoConnect) {
        this.mt5Service = mt5Service;
        this.isConnected = autoConnect;
    }

    /**
     * Checks if the MT5 bridge connection is toggled active.
     *
     * @return true if connected, false otherwise
     */
    public boolean isConnected() {
        // Optionally: you could perform a real-time check here too,
        // but for UI stability, we use the isConnected flag which is toggled by Admin.
        return isConnected;
    }

    /**
     * Attempts to connect to MT5 by verifying the bridge.
     * The goal of this method is to ping the bridge health and update the connected flag.
     *
     * @return true if bridge is reachable and flag is updated, false otherwise
     */
    public boolean attemptConnect() {
        if (mt5Service.checkHealth()) {
            this.isConnected = true;
            return true;
        }
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/MT5JavaTradeWriter.java | סוג: class |
אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 47

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import com.sun.jna.Library;
import com.sun.jna.Native;

import java.io.OutputStream;
import java.io.PrintWriter;
import java.net.Socket;

public class MT5JavaTradeWriter {

    private static final String MT5_HOST = "127.0.0.1";
    private static final int MT5_PORT = 5555;

    public interface TradeDLL extends Library {
        TradeDLL INSTANCE = Native.load("TradeBridge2", TradeDLL.class);
        boolean GetLastPositions(byte[] buffer, int maxLen);
    }

    public static String sendTradeToMT5(String symbol, String action, double entry, double tp, double sl, double lot) {
        String response;

        try (Socket socket = new Socket(MT5_HOST, MT5_PORT);
            OutputStream output = socket.getOutputStream();
            PrintWriter writer = new PrintWriter(output, true)) {

            String cleanSymbol = symbol.trim();
            String cleanAction = action.trim();

            String command = String.format(
                "TRADE|%s|%s|%.5f|%.5f|%.5f|%.2f",
                cleanSymbol + ".m", cleanAction, entry, tp, sl, lot
            );

            writer.println(command);
            System.out.println("✅ Trade signal sent via socket: " + command);
            response = "SUCCESS|Trade sent via socket";

        } catch (Exception e) {
            System.err.println("❌ Socket communication failed");
            e.printStackTrace();
            response = "❌ ERROR|Socket failed: " + e.getMessage();
        }

        return response;
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/MarginLoanLedger.java | סוג: class |
אנוטציות:

@Column, @ManyToOne, @CreationTimestamp, @GeneratedValue, @Entity, @Table, @JoinColumn, @Id
שורות קוד: 105

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.math.BigDecimal;
import java.time.Instant;
import org.hibernate.annotations.CreationTimestamp;

/**
 * A single entry in the margin credit line ledger: a borrow, a repayment,
 * an interest charge, or a forced liquidation event on a trading account.
 */
@Entity
@Table(name = "margin_loan_ledger")
public class MarginLoanLedger {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "trading_account_id", nullable = false)
    private TradingAccount tradingAccount;

    @Column(name = "entry_type", nullable = false, length = 20)
    private String entryType; // BORROW, REPAY, INTEREST, LIQUIDATION

    @Column(nullable = false, precision = 18, scale = 8)
    private BigDecimal amount;

    @Column(name = "borrowed_after", nullable = false, precision = 18, scale = 8)
    private BigDecimal borrowedAfter;

    @Column(name = "balance_after", nullable = false, precision = 18, scale = 8)
    private BigDecimal balanceAfter;

    @Column(length = 255)
    private String note;

    @CreationTimestamp
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    public Long getId() {

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/Notification.java | סוג: class |
אנוטציות:

@Column, @ManyToOne, @CreationTimestamp, @GeneratedValue, @JsonIgnore, @Entity, @Table,
@JoinColumn, @Id

| שורות קוד: 93

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import com.fasterxml.jackson.annotation.JsonIgnore;
import com.brokerui.user.AppUser;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.time.Instant;
import org.hibernate.annotations.CreationTimestamp;

@Entity
@Table(name = "notification")
public class Notification {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @JsonIgnore
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private AppUser user;

    @Column(name = "notif_type", nullable = false, length = 32)
    private String notifType;

    @Column(nullable = false, length = 128)
    private String title;

    @Column(nullable = false, columnDefinition = "TEXT")
    private String body;

    @Column(name = "read_at")
    private Instant readAt;

    @CreationTimestamp
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    public Long getId() {
        return id;
    }

    public AppUser getUser() {
        return user;
    }

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/Position.java | סוג: class | אנוטציות: @Column, @ManyToOne, @GeneratedValue, @JsonIgnore, @Entity, @Table, @JoinColumn, @UpdateTimestamp, @Id | שורות קוד: 147

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.math.BigDecimal;
import java.time.Instant;
import org.hibernate.annotations.UpdateTimestamp;

@Entity
@Table(name = "position")
public class Position {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @JsonIgnore
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "trading_account_id", nullable = false)
    private TradingAccount tradingAccount;

    @Column(name = "symbol_code", nullable = false, length = 32)
    private String symbolCode;

    @Column(nullable = false, length = 8, columnDefinition = "VARCHAR(8) DEFAULT 'LONG'")
    private String side = "LONG";

    @Column(nullable = false, precision = 18, scale = 8)
    private BigDecimal quantity = BigDecimal.ZERO;

    @Column(name = "avg_price", precision = 18, scale = 8)
    private BigDecimal avgPrice;

    @Column(name = "realized_pnl", nullable = false, precision = 18, scale = 8)
    private BigDecimal realizedPnl = BigDecimal.ZERO;

    @Column(name = "unrealized_pnl", nullable = false, precision = 18, scale = 8)
    private BigDecimal unrealizedPnl = BigDecimal.ZERO;

    @Column(name = "opened_at")
    private Instant openedAt;

    @UpdateTimestamp

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/Symbol.java | סוג: class | אנוטציות: @Column, @CreationTimestamp, @GeneratedValue, @Entity, @Table, @Id | שורות קוד: 97

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import java.time.Instant;
import org.hibernate.annotations.CreationTimestamp;

@Entity
@Table(name = "symbol")
public class Symbol {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 32)
    private String code;

    @Column(nullable = false, length = 16)
    private String kind;

    @Column(name = "base_currency", length = 8)
    private String baseCurrency;

    @Column(name = "quote_currency", length = 8)
    private String quoteCurrency;

    @Column(name = "price_decimals", nullable = false)
    private int priceDecimals = 2;

    @Column(nullable = false)
    private boolean enabled = true;

    @CreationTimestamp
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    public Long getId() {
        return id;
    }

    public String getCode() {
        return code;
    }

    public void setCode(String code) {
        this.code = code;
    }

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/TradeFill.java | סוג: class | אנוטציות: @Column, @ManyToOne, @CreationTimestamp, @GeneratedValue, @Entity, @Table, @JoinColumn, @Id | שורות קוד: 91

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.math.BigDecimal;
import java.time.Instant;
import org.hibernate.annotations.CreationTimestamp;

@Entity
@Table(name = "trade_fill")
public class TradeFill {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "order_id", nullable = false)
    private BrokerOrder order;

    @Column(nullable = false, precision = 18, scale = 8)
    private BigDecimal price;

    @Column(nullable = false, precision = 18, scale = 8)
    private BigDecimal quantity;

    @Column(nullable = false, precision = 18, scale = 8)
    private BigDecimal fee = BigDecimal.ZERO;

    @Column(length = 8)
    private String liquidity;

    @CreationTimestamp
    @Column(name = "executed_at", nullable = false, updatable = false)
    private Instant executedAt;

    public Long getId() {
        return id;
    }

    public BrokerOrder getOrder() {
        return order;
    }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/TradingAccount.java | סוג: class |
אנוטציות:

@Column, @ManyToOne, @CreationTimestamp, @GeneratedValue, @Entity, @Table, @JoinColumn,
@UpdateTimestamp, @Id

| שורות קוד: 219

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import com.brokerui.user.AppUser;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.math.BigDecimal;
import java.time.Instant;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

@Entity
@Table(name = "trading_account")
public class TradingAccount {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private AppUser user;

    @Column(name = "account_type", nullable = false, length = 16)
    private String accountType = "DEMO";

    @Column(nullable = false, length = 8)
    private String currency = "USD";

    @Column(nullable = false)
    private int leverage = 100;

    @Column(nullable = false, length = 16)
    private String status = "ACTIVE";

    @Column(nullable = false, precision = 18, scale = 8)
    private BigDecimal balance = BigDecimal.ZERO;

    @Column(nullable = false, precision = 18, scale = 8)
    private BigDecimal equity = BigDecimal.ZERO;

    @Column(name = "margin_used", nullable = false, precision = 18, scale = 8)
    private BigDecimal marginUsed = BigDecimal.ZERO;

    @Column(name = "free_margin", nullable = false, precision = 18, scale = 8)

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/UserPreference.java | סוג: class |
אנוטציות: @Column, @ManyToOne, @GeneratedValue, @Entity, @Table, @JoinColumn, @UpdateTimestamp, @Id | שורות קוד: 49

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.broker;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.time.Instant;
import org.hibernate.annotations.UpdateTimestamp;
import com.brokerui.user.AppUser;

@Entity
@Table(name = "user_preference")
public class UserPreference {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "user_id", nullable = false)
    private AppUser user;

    @Column(name = "pref_key", nullable = false, length = 64)
    private String prefKey;

    @Column(name = "pref_value")
    private String prefValue;

    @UpdateTimestamp
    @Column(name = "updated_at", nullable = false)
    private Instant updatedAt;

    public Long getId() { return id; }

    public AppUser getUser() { return user; }
    public void setUser(AppUser user) { this.user = user; }

    public String getPrefKey() { return prefKey; }
    public void setPrefKey(String prefKey) { this.prefKey = prefKey; }

    public String getPrefValue() { return prefValue; }
    public void setPrefValue(String prefValue) { this.prefValue = prefValue; }

    public Instant getUpdatedAt() { return updatedAt; }
}

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/market/CandleBar.java | סוג: record | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 7

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```
package com.brokerui.market;

/**
 * Data transfer record representing a single historical candlestick bar.
 * Holds open, high, low, close, and timestamp.
 */
public record CandleBar(long time, double open, double high, double low, double close) {}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/sim/SimTrade.java | סוג: class | אנוטציות:
@Column, @GeneratedValue, @Entity, @Table, @Id | שורות קוד: 61

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.sim;

import java.time.Instant;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "sim_trade")
public class SimTrade {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 32)
    private String symbol;

    @Column(nullable = false)
    private Integer quantity;

    @Column(name = "buyer_label", nullable = false, length = 64)
    private String buyerLabel;

    @Column(name = "created_at", nullable = false, updatable = false, insertable = false)
    private Instant createdAt;

    public Long getId() {
        return id;
    }

    public String getSymbol() {
        return symbol;
    }

    public void setSymbol(String symbol) {
        this.symbol = symbol;
    }

    public Integer getQuantity() {
        return quantity;
    }

    public void setQuantity(Integer quantity) {
        this.quantity = quantity;
    }

    public String getBuyerLabel() {

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/sim/SimTradeDto.java | סוג: record | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 10

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```
package com.brokerui.sim;

import java.time.Instant;

public record SimTradeDto(Long id, String symbol, int quantity, String buyerLabel, Instant createdAt) {
    public static SimTradeDto from(SimTrade t) {
        return new SimTradeDto(
            t.getId(), t.getSymbol(), t.getQuantity(), t.getBuyerLabel(), t.getCreatedAt());
    }
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/user/AppUser.java | סוג: class | אנוטציות: @Column, @Enumerated, @CreationTimestamp, @GeneratedValue, @JsonIgnore, @Entity, @Table, @Id | שורות קוד: 113

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```

package com.brokerui.user;

import com.fasterxml.jackson.annotation.JsonIgnore;
import java.time.Instant;
import org.hibernate.annotations.CreationTimestamp;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "app_user")
public class AppUser {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(name = "display_name", nullable = false)
    private String displayName;

    @Column(name = "password_hash")
    @JsonIgnore
    private String passwordHash;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 32)
    private UserRole role = UserRole.USER;

    @CreationTimestamp
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    @Column(name = "banned", nullable = false)
    private boolean banned = false;

    @Column(name = "banned_at")
    private Instant bannedAt;

    @Column(name = "banned_reason", length = 500)
    private String bannedReason;

    public Long getId() {
        return id;
    }

```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/user/UserDto.java | סוג: record | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 26

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```
package com.brokerui.user;

import java.time.Instant;

public record UserDto(
    Long id,
    String email,
    String displayName,
    String role,
    Instant createdAt,
    boolean banned,
    Instant bannedAt,
    String bannedReason) {
    public static UserDto from(AppUser u) {
        return new UserDto(
            u.getId(),
            u.getEmail(),
            u.getDisplayName(),
            u.getRole().name(),
            u.getCreatedAt(),
            u.isBanned(),
            u.getBannedAt(),
            u.getBannedReason());
    }
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/user/UserRole.java | סוג: enum | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 6

תפקיד הישות במערכת: ייצוג טבלת מסד נתונים (JPA Entity / Data Model). משמשת לשמירת נתונים יציבים (משתמשים, חשבונות, הוראות, פוזיציות, יומנים).

```
package com.brokerui.user;

public enum UserRole {
    USER,
    ADMIN
}
```

פרק 10: מילון מפורט - JPA Repositories (13 מחלקות)

מאגר נתונים JPA

com.brokerui.broker.AccountTransactionRepository

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/AccountTransactionRepository.java | סוג: interface | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 9

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאלות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.broker;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AccountTransactionRepository extends JpaRepository<AccountTransaction, Long> {
    List<AccountTransaction> findByTradingAccountIdOrderByCreatedAtDesc(Long tradingAccountId);
}
```

מאגר נתונים JPA

com.brokerui.broker.AuditLogRepository

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/AuditLogRepository.java | סוג: interface | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 6

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאלות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.broker;

import org.springframework.data.jpa.repository.JpaRepository;

public interface AuditLogRepository extends JpaRepository<AuditLog, Long> {}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/BrokerOrderRepository.java | סוג: interface | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 17

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאלות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.broker;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface BrokerOrderRepository extends JpaRepository<BrokerOrder, Long> {
    List<BrokerOrder> findByTradingAccountIdOrderByCreatedAtDesc(Long tradingAccountId);

    List<BrokerOrder> findByTradingAccountIdAndStatusOrderByFilledAtDesc(Long tradingAccountId, String status);

    List<BrokerOrder> findByTradingAccountIdAndStatusOrderByCreatedAtDesc(Long tradingAccountId, String status);

    List<BrokerOrder> findTop50ByStatusOrderByCreatedAtAsc(String status);

    long countBySymbolCodeAndSideAndStatus(String symbolCode, String side, String status);
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/KycCaseRepository.java | סוג: interface | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 9

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאילתות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.broker;

import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface KycCaseRepository extends JpaRepository<KycCase, Long> {
    Optional<KycCase> findById(Long userId);
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/MarginLoanLedgerRepository.java | סוג: interface | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 10

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאילתות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.broker;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface MarginLoanLedgerRepository extends JpaRepository<MarginLoanLedger, Long> {
    List<MarginLoanLedger> findByTradingAccountIdOrderByCreatedAtDesc(Long tradingAccountId);

    List<MarginLoanLedger> findAllOrderByCreatedAtDesc();
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/NotificationRepository.java | סוג: interface | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 9

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאלות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.broker;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface NotificationRepository extends JpaRepository<Notification, Long> {
    List<Notification> findTop20ByUserIdOrderByCreatedAtDesc(Long userId);
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/PositionRepository.java | סוג: interface | אנוטציות: @Param, @Lock, @Query | שורות קוד: 40

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאילתות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository (save, findById, findAll, delete)

```
package com.brokerui.broker;

import jakarta.persistence.LockModeType;
import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Lock;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface PositionRepository extends JpaRepository<Position, Long> {

    List<Position> findByTradingAccountIdOrderByUpdatedAtDesc(Long tradingAccountId);

    Optional<Position> findByTradingAccountIdAndSymbolCode(Long tradingAccountId, String symbolCode);

    Optional<Position> findByTradingAccountIdAndSymbolCodeAndSide(
        Long tradingAccountId, String symbolCode, String side);

    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT p FROM Position p WHERE p.tradingAccount.id = :accountId AND p.symbolCode = :symbolCode")
    Optional<Position> findByTradingAccountIdAndSymbolCodeAndSideForUpdate(
        @Param("accountId") Long accountId, @Param("symbolCode") String symbolCode, @Param("side") String side);

    /**
     * Pessimistic write lock on a single position - used when placing a SELL order
     * or executing order matching where side isn't explicitly known.
     */
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT p FROM Position p WHERE p.tradingAccount.id = :accountId AND p.symbolCode = :symbolCode")
    Optional<Position> findByTradingAccountIdAndSymbolCodeForUpdate(
        @Param("accountId") Long accountId, @Param("symbolCode") String symbolCode);

    /**
     * Pessimistic write lock by position id - used when closing a position.
     */
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT p FROM Position p WHERE p.id = :id")
    Optional<Position> findByIdForUpdate(@Param("id") Long id);
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/SymbolRepository.java | סוג: interface
 | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 12

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאלות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.broker;

import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface SymbolRepository extends JpaRepository<Symbol, Long> {
    List<Symbol> findByEnabledTrueOrderByKindAscCodeAsc();

    Optional<Symbol> findByCode(String code);
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/TradeFillRepository.java | סוג: interface
 | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 9

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאלות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.broker;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface TradeFillRepository extends JpaRepository<TradeFill, Long> {
    List<TradeFill> findByOrderId(Long orderId);
    List<TradeFill> findByOrderIdOrderByExecutedAtDesc(Long orderId);
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/TradingAccountRepository.java | סוג: interface | אנושיות: @Lock, @Param, @Transactional, @Query | שורות קוד: 29

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאלות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository (save, findById, findAll, delete)

```
package com.brokerui.broker;

import jakarta.persistence.LockModeType;
import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Lock;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;

public interface TradingAccountRepository extends JpaRepository<TradingAccount, Long> {

    List<TradingAccount> findByIdOrderByIdAsc(Long userId);

    Optional<TradingAccount> findFirstByIdOrderByIdAsc(Long userId);

    /**
     * Pessimistic write lock - use inside @Transactional methods that modify balance
     * to prevent lost-update race conditions when multiple requests hit the same account.
     */
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("SELECT ta FROM TradingAccount ta WHERE ta.id = :id")
    Optional<TradingAccount> findByIdForUpdate(@Param("id") Long id);

    /** Accounts that currently owe money on the margin credit line. */
    List<TradingAccount> findByBorrowedBalanceGreaterThan(BigDecimal threshold);
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/broker/UserPreferenceRepository.java | סוג: interface | אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 11

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאלות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.broker;

import com.brokerui.user.AppUser;
import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserPreferenceRepository extends JpaRepository<UserPreference, Long> {
    Optional<UserPreference> findByUserAndPrefKey(AppUser user, String prefKey);
    List<UserPreference> findByUser(AppUser user);
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/sim/SimTradeRepository.java | סוג: interface
| אנוניזציות: @return | שורות קוד: 17

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאלות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.sim;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

/**
 * Repository interface for managing SimTrade database records.
 */
public interface SimTradeRepository extends JpaRepository<SimTrade, Long> {
    /**
     * Retrieves the top 200 simulation trades sorted by ID in descending order.
     * The goal of this method is to load recent simulation history for monitoring.
     *
     * @return list of recent simulation trades
     */
    List<SimTrade> findTop200ByOrderByIdDesc();
}
```

נתיב קובץ: backend/backend/src/main/java/com/brokerui/user/AppUserRepository.java | סוג: interface
| אנוטציות: ללא אנוטציות מיוחדות | שורות קוד: 17

תפקיד המאגר במערכת: Spring Data JPA Repository המספק גישה נוחה לשליפה, עדכון וביצוע שאילתות מול מסד הנתונים.

שאלות ושיטות מותאמות: תמיכה מלאה במתודות JpaRepository הסטנדרטיות (save, findById, findAll, delete)

```
package com.brokerui.user;

import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AppUserRepository extends JpaRepository<AppUser, Long> {
    Optional<AppUser> findByEmail(String email);

    Optional<AppUser> findByEmailIgnoreCase(String email);

    boolean existsByEmail(String email);

    boolean existsByEmailIgnoreCase(String email);

    boolean existsByRole(UserRole role);
}
```


פרק 11: מילון מפורט - Frontend Components, Hooks &

ממשק אדמין / HTML

AdminDashboard_Local.html

נתיב קובץ: AdminDashboard_Local.html | סיומת: .html | שורות קוד: 1181

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `deleteUser()`, `q()`, `openEditBalance()`, `refreshData()`, `userFilter()`, `renderTransactionsTable()`, `renderCreditLineTable()`, `restartServer()`, `renderAuditTable()`, `postData()`, `banUser()`, `runInterestNow()`

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Broker Admin | Local Controller</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&display=swap"
  <style>
    :root {
      --bg: #0a0c10;
      --bg-gradient: radial-gradient(circle at top right, #131a26 0%, #0a0c10 100%);
      --card-bg: rgba(13, 17, 23, 0.7);
      --card-shadow: 0 8px 24px rgba(0, 0, 0, 0.4);
      --primary: #00c8ff;
      --primary-glow: rgba(0, 200, 255, 0.4);
      --text: #f0f6fc;
      --text-muted: #8b949e;
      --border: rgba(255, 255, 255, 0.1);
      --success: #23d160;
      --danger: #f85149;
      --warning: #e3b341;
      --sidebar-w: 260px;
    }

    * { box-sizing: border-box; }
    body {
      margin: 0;
      font-family: 'Outfit', sans-serif;
      background: var(--bg-gradient);
      background-attachment: fixed;
      color: var(--text);
      display: flex;
      height: 100vh;
      overflow: hidden;
    }

    aside {
      width: var(--sidebar-w);
      background: rgba(13, 17, 23, 0.85);
      backdrop-filter: blur(20px);
      border-right: 1px solid var(--border);
      display: flex;
      flex-direction: column;

```

נתיב קובץ: UI-react/eslint.config.js | סיומת: .js | שורות קוד: 34

תפקיד הקובץ במערכת: רכיב UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
import js from '@eslint/js';
import tsParser from '@typescript-eslint/parser';
import tsPlugin from '@typescript-eslint/eslint-plugin';
import reactHooks from 'eslint-plugin-react-hooks';
import reactRefresh from 'eslint-plugin-react-refresh';
import prettierConfig from 'eslint-config-prettier';
import globals from 'globals';

export default [
  { ignores: ['dist', 'node_modules'] },
  js.configs.recommended,
  {
    files: ['**/*.ts', '**/*.tsx'],
    languageOptions: {
      parser: tsParser,
      parserOptions: { project: './tsconfig.json', ecmaFeatures: { jsx: true } },
      globals: { ...globals.browser, ...globals.es2020 },
    },
    plugins: {
      '@typescript-eslint': tsPlugin,
      'react-hooks': reactHooks,
      'react-refresh': reactRefresh,
    },
    rules: {
      ...tsPlugin.configs.recommended.rules,
      ...reactHooks.configs.recommended.rules,
      'react-refresh/only-export-components': ['warn', { allowConstantExport: true }],
      '@typescript-eslint/no-explicit-any': 'warn',
      '@typescript-eslint/no-unused-vars': ['error', { argsIgnorePattern: '^_' }],
      'no-console': ['warn', { allow: ['warn', 'error'] }],
    },
  },
  prettierConfig,
];
```

נתיב קובץ: UI-react/package-lock.json | סיומת: .json | שורות קוד: 3094

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```

{
  "name": "broker-ui-react",
  "version": "0.1.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "broker-ui-react",
      "version": "0.1.0",
      "dependencies": {
        "@hookform/resolvers": "^5.4.0",
        "@stomp/stompjs": "^7.3.0",
        "@tanstack/react-query": "^5.100.14",
        "@tanstack/react-query-devtools": "^5.100.14",
        "lightweight-charts": "^5.2.0",
        "react": "^18.3.1",
        "react-dom": "^18.3.1",
        "react-hook-form": "^7.76.1",
        "react-router-dom": "^6.26.2",
        "sockjs-client": "^1.6.1"
      },
      "devDependencies": {
        "@eslint/js": "^10.0.1",
        "@types/react": "^18.3.12",
        "@types/react-dom": "^18.3.1",
        "@types/sockjs-client": "^1.5.4",
        "@typescript-eslint/eslint-plugin": "^8.60.0",
        "@typescript-eslint/parser": "^8.60.0",
        "@vitejs/plugin-react": "^4.3.3",
        "eslint": "^10.4.0",
        "eslint-config-prettier": "^10.1.8",
        "eslint-plugin-react-hooks": "^7.1.1",
        "eslint-plugin-react-refresh": "^0.5.2",
        "globals": "^17.6.0",
        "prettier": "^3.8.3",
        "typescript": "^5.6.3",
        "vite": "^5.4.10"
      }
    },
    "node_modules/@babel/code-frame": {
      "version": "7.29.0",
      "resolved": "https://registry.npmjs.org/@babel/code-frame/-/code-frame-7.29.0.tgz",
      "integrity": "sha512-01181183118281337302d39418e23d7518a41d2f",
      "dev": true,
      "dependencies": {

```

נתיב קובץ: UI-react/package.json | סיומת: .json | שורות קוד: 41

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
{
  "name": "broker-ui-react",
  "private": true,
  "version": "0.1.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview",
    "lint": "eslint . --ext ts,tsx --report-unused-disable-directives --max-warnings 0"
  },
  "dependencies": {
    "@hookform/resolvers": "^5.4.0",
    "@stomp/stompjs": "^7.3.0",
    "@tanstack/react-query": "^5.100.14",
    "@tanstack/react-query-devtools": "^5.100.14",
    "lightweight-charts": "^5.2.0",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-hook-form": "^7.76.1",
    "react-router-dom": "^6.26.2",
    "sockjs-client": "^1.6.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^18.3.12",
    "@types/react-dom": "^18.3.1",
    "@types/sockjs-client": "^1.5.4",
    "@typescript-eslint/eslint-plugin": "^8.60.0",
    "@typescript-eslint/parser": "^8.60.0",
    "@vitejs/plugin-react": "^4.3.3",
    "eslint": "^10.4.0",
    "eslint-config-prettier": "^10.1.8",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.2",
    "globals": "^17.6.0",
    "prettier": "^3.8.3",
    "typescript": "^5.6.3",
    "vite": "^5.4.10"
  }
}
```

נתיב קובץ: UI-react/src/App.tsx | סיומת: .tsx | שורות קוד: 76

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `App()`

```

import { HashRouter, Navigate, Route, Routes } from 'react-router-dom';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { ReactQueryDevtools } from '@tanstack/react-query-devtools';
import { I18nProvider } from '@/hooks/useI18n';
import { ToastProvider } from '@/hooks/useToast';
import { AuthProvider } from '@/hooks/useAuth';
import { ProtectedRoute } from '@/components/ProtectedRoute';
import { PreferenceSync } from '@/components/PreferenceSync';
import { ErrorBoundary } from '@/components/ErrorBoundary';
import { LoginPage } from '@/pages/LoginPage';
import { RegisterPage } from '@/pages/RegisterPage';
import { DemoRegisterPage } from '@/pages/DemoRegisterPage';
import { LandingPage } from '@/pages/LandingPage';
import { AccountPage } from '@/pages/AccountPage';
import { DashboardPage } from '@/pages/DashboardPage';
import { SettingsPage } from '@/pages/SettingsPage';
import { PositionsPage } from '@/pages/PositionsPage';
import { HistoryPage } from '@/pages/HistoryPage';
import { FinancePage } from '@/pages/FinancePage';
import { ChartsPage } from '@/pages/ChartsPage';
import { AnalyzerPage } from '@/pages/AnalyzerPage';
import { PlaceholderPage } from '@/pages/PlaceholderPage';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: 1,
      staleTime: 30_000,
    },
  },
});

/**
 * Main application React entry component.
 * Registers context providers (QueryClientProvider, ErrorBoundary, I18nProvider, ToastProvider, AuthProvider),
 * configures page routes mapping, and enables React Query Devtools context.
 *
 * @returns Root application layout element with routing
 */
export function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <ErrorBoundary>
        <I18nProvider>
          <ToastProvider>

```

נתיב קובץ: UI-react/src/components/AuthHeader.tsx | סיומת: .tsx | שורות קוד: 47

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: AuthHeader()

```
import { Link } from 'react-router-dom';
import type { ReactNode } from 'react';
import type { Lang } from '@types/api';
import { useI18n } from '@hooks/useI18n';

const LANG_OPTIONS: ReadonlyArray<{ code: Lang; label: string }> = [
  { code: 'en', label: 'EN' },
  { code: 'ru', label: 'RU' },
  { code: 'he', label: 'HE' },
];

interface Props {
  children?: ReactNode;
}

/**
 * Header component for authentication views (login/register).
 * Provides logo brand linkage and general language localization switcher buttons.
 *
 * @param props children layout elements
 * @returns Top navigation header component
 */
export function AuthHeader({ children }: Props) {
  const { lang, setLang } = useI18n();

  return (
    <header className="top-nav">
      <Link to="/landing" className="brand-logo-link" aria-label="Broker App">
        
      </Link>
      {children}
      <div style={{ marginInlineStart: 'auto', display: 'flex', gap: 4 }}>
        {LANG_OPTIONS.map((opt) => (
          <button
            key={opt.code}
            type="button"
            className={`btn ${lang === opt.code ? 'btn-primary' : 'btn-outline'}`}
            style={{ padding: '4px 10px', fontSize: '0.8rem' }}
            onClick={() => setLang(opt.code)}
          >
            {opt.label}
          </button>
        ))}
      </div>
    </header>
  );
}
```

נתיב קובץ: UI-react/src/components/BackPageHeader.tsx | סיומת: .tsx | שורות קוד: 35

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: BackPageHeader()

```
import { Link } from 'react-router-dom';
import { useI18n } from '@hooks/useI18n';

interface Props {
  titleKey: string;
  backTo?: string;
}

/**
 * Centered page header component featuring a Back navigation link.
 * Used across sub-screens like Charts, Finance, Settings, and History.
 *
 * @param props titleKey to resolve and optional backTo navigation route path
 * @returns Centered header component with Back action
 */
export function BackPageHeader({ titleKey, backTo = '/dashboard' }: Props) {
  const { t } = useI18n();

  return (
    <header className="top-nav header-centered">
      <div className="header-leading">
        <Link to={backTo} className="brand-logo-link" aria-label="Broker App">
          
        </Link>
        <Link to={backTo} className="btn btn-outline header-back" aria-label={t('common.back')}>
          <svg width="16" height="16" fill="none" stroke="currentColor" strokeWidth="2.5" viewBox="0 0 16 16">
            <path strokeLinecap="round" strokeLinejoin="round" d="M15 19l-7-7 7-7" />
          </svg>
          <span>{t('common.back')}</span>
        </Link>
      </div>
      <h2 className="header-page-title">{t(titleKey)}</h2>
    </header>
  );
}
```

נתיב קובץ: UI-react/src/components/DashboardHeader.tsx | סיומת: .tsx | שורות קוד: 97

תפקיד הקובץ במערכת: רכיב UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `DashboardHeader()`, `handleLangChange()`

```
import { Link, useLocation } from 'react-router-dom';
import { useI18n } from '@/hooks/useI18n';
import { useSavePreferences } from '@/hooks/useApi';
import type { Lang } from '@/types/api';

const NAV_LINKS = [
  { to: '/positions', key: 'nav.positions' },
  { to: '/charts', key: 'nav.charts' },
  { to: '/analyzer', key: 'nav.analyzer' },
  { to: '/finance', key: 'nav.finance' },
  { to: '/history', key: 'nav.history' },
  { to: '/account', key: 'nav.account' },
  { to: '/settings', key: 'nav.settings' },
] as const;

const LANG_OPTIONS: ReadonlyArray<{ code: Lang; label: string }> = [
  { code: 'en', label: 'EN' },
  { code: 'ru', label: 'RU' },
  { code: 'he', label: 'HE' },
];

interface Props {
  onLogout: () => void;
  loggingOut?: boolean;
}

/**
 * Primary header navigation for authenticated users on the dashboard and main pages.
 * Displays navigation links, language selector buttons, and the sign out action.
 *
 * @param props onLogout function and loggingOut indicator flag
 * @returns Primary authenticated layout top navigation bar
 */
export function DashboardHeader({ onLogout, loggingOut }: Props) {
  const { t, lang, setLang } = useI18n();
  const { pathname } = useLocation();
  const { mutate: savePrefs } = useSavePreferences();

  /**
   * Updates language preference in app state and saves it to the backend database.
   *
   * @param code target language code (e.g. en, he, ru)
   */
  const handleLangChange = (code: Lang) => {
    setLang(code);
  }
}
```

נתיב קובץ: UI-react/src/components/ErrorBoundary.tsx | סימנת: .tsx | שורות קוד: 113

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
import { Component } from 'react';
import type { ReactNode, ErrorInfo } from 'react';

interface Props {
  children: ReactNode;
}

interface State {
  hasError: boolean;
  error: Error | null;
}

/**
 * React ErrorBoundary component that catches runtime rendering errors in its children tree.
 * Prevents application crashes and renders an elegant fallback UI detailing the exception.
 */
export class ErrorBoundary extends Component<Props, State> {
  /**
   * Initializes the ErrorBoundary component state.
   */
  constructor(props: Props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  /**
   * Derives error state updates dynamically when a child throws an exception.
   *
   * @param error caught runtime Error object
   * @returns updated state object
   */
  static getDerivedStateFromError(error: Error): State {
    return { hasError: true, error };
  }

  /**
   * Lifecycle hook to log caught exception details.
   *
   * @param error caught error
   * @param info error metadata (component stack trace info)
   */
  componentDidCatch(error: Error, info: ErrorInfo) {
    console.error('[ErrorBoundary]', error, info.componentStack);
  }
}
```

נתיב קובץ: UI-react/src/components/LoanOfferModal.tsx | סיומת: .tsx | שורות קוד: 125

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `dailyRatePct()`

```
import React from 'react';

export interface LoanOfferDetails {
  shortfall: number;
  required: number;
  cashBalance: number;
  creditLimit: number;
  dailyInterestRate: number;
  symbol: string;
  side: string;
  onConfirm: () => void;
  onCancel: () => void;
}

export const LoanOfferModal: React.FC<LoanOfferDetails> = ({
  shortfall,
  required,
  cashBalance,
  creditLimit,
  dailyInterestRate,
  symbol,
  side,
  onConfirm,
  onCancel,
}) => {
  const dailyRatePct = (dailyInterestRate * 100).toFixed(1);

  return (
    <div
      style={{
        position: 'fixed',
        top: 0,
        left: 0,
        right: 0,
        bottom: 0,
        backgroundColor: 'rgba(0, 0, 0, 0.75)',
        backdropFilter: 'blur(4px)',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'center',
        zIndex: 9999,
        padding: 20,
      }}
    >
      <div
```

נתיב קובץ: UI-react/src/components/NNAdvisorPanel.tsx | סיומת: .tsx | שורות קוד: 206

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `NNAdvisorPanel()`, `imbalance()`, `fetchLivePrediction()`

```
import { useEffect, useState } from 'react';

interface NNAdvisorPanelProps {
  symbol: string;
  volume: number;
  currentPrice: number;
}

/**
 * AI Neural Network Smart Routing Advisor panel.
 * Posts normalized book/market parameters to the python predictor service (port 3005)
 * and displays recommended routing (Internal Crossing vs. External MT5) with estimated savings.
 *
 * @param props symbol, active input volume quantity, and current price quote
 * @returns Advisor panel layout or null if inputs are not positive
 */
export function NNAdvisorPanel({ symbol, volume, currentPrice }: NNAdvisorPanelProps) {
  const [prediction, setPrediction] = useState<{
    matchProb: number;
    expectedSavings: number;
    routeRecommendation: number;
  } | null>(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let active = true;
    async function fetchLivePrediction() {
      setLoading(true);
      setError(null);
      try {
        let scaledPrice = currentPrice;
        if (currentPrice > 0) {
          const log10 = Math.log10(currentPrice);
          const exp = Math.round(log10) - 2;
          scaledPrice = currentPrice / Math.pow(10, exp);
        }
        const buyQtyNorm = volume / 100.0;
        const sellQtyNorm = 0.45;
        const spreadNorm = Math.min((scaledPrice * 0.00015) / 1.0, 1.0);
        const imbalance = (volume - 45) / (volume + 45);
        const midPriceNorm = Math.min(scaledPrice / 200.0, 1.0);
        const bookDepthBuy = 0.6;
        const bookDepthSell = 0.5;
        const historicalMatchRate = 0.72;
```

נתיב קובץ: UI-react/src/components/PreferenceSync.tsx | סיומת: .tsx | שורות קוד: 67

תפקיד הקובץ במערכת: רכיב UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `applyTheme()`, `PreferenceSync()`

```
import { useEffect } from 'react';
import type { ReactNode } from 'react';
import { useAuth } from '@/hooks/useAuth';
import { useI18n } from '@/hooks/useI18n';
import { persistLang } from '@/lib/i18n';
import { usePreferences } from '@/hooks/useApi';
import type { Lang } from '@/types/api';

type Theme = 'dark' | 'light' | 'system';
const ALLOWED_LANGS: ReadonlySet<string> = new Set(['en', 'ru', 'he']);
const ALLOWED_THEMES: ReadonlySet<string> = new Set(['dark', 'light', 'system']);

/**
 * Helper utility to programmatically apply the selected UI theme to documentElement attributes.
 * Also persists the choice to localStorage.
 *
 * @param theme target theme style choice
 */
function applyTheme(theme: Theme) {
  const root = document.documentElement;
  if (theme === 'system') {
    const prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
    root.setAttribute('data-theme', prefersDark ? 'dark' : 'light');
  } else {
    root.setAttribute('data-theme', theme);
  }
  localStorage.setItem('theme', theme);
}

/**
 * Sync component that loads user preferences from the DB once after successful authentication
 * and applies language + theme globally. This ensures settings are consistent
 * across devices and sessions.
 *
 * @param props children layout elements
 * @returns Sync context wrapper element
 */
export function PreferenceSync({ children }: { children: ReactNode }) {
  const { status } = useAuth();
  const { setLang } = useI18n();

  // Reuse the same query key as usePreferences() so both share the same cache entry.
  // Guard with status so we don't fire a 401 request before login.
  const { data: prefs } = usePreferences(status === 'authenticated');
```

נתיב קובץ: UI-react/src/components/ProtectedRoute.tsx | סיומת: .tsx | שורות קוד: 52

תפקיד הקובץ במערכת: רכיב UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: ProtectedRoute()

```
import { Navigate, useLocation } from 'react-router-dom';
import type { ReactNode } from 'react';
import { useAuth } from '@hooks/useAuth';

interface Props {
  children: ReactNode;
}

/**
 * Route protector wrapper component.
 * Validates active session status and redirects unauthenticated users to the login route path.
 *
 * @param props children layout elements to guard
 * @returns Guarded children layout or spinner loader
 */
export function ProtectedRoute({ children }: Props) {
  const { status } = useAuth();
  const location = useLocation();

  if (status === 'loading') {
    return (
      <div
        style={{
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          height: '100vh',
          flexDirection: 'column',
          gap: 16,
        }}
      >
        <div
          style={{
            width: 40,
            height: 40,
            border: '3px solid var(--border-light)',
            borderTopColor: 'var(--primary)',
            borderRadius: '50%',
            animation: 'spin 0.8s linear infinite',
          }}
        />
        <span style={{ color: 'var(--text-secondary)', fontSize: '0.9rem' }}>Loading...</span>
      </div>
    );
  }
}
```

נתיב קובץ: UI-react/src/components/Skeleton.tsx | סיומת: .tsx | שורות קוד: 65

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `SkeletonRow()`, `Skeleton()`, `SkeletonCard()`

◀ ▶

נתיב קובץ: UI-react/src/hooks/useApi.ts | סיומת: .ts | שורות קוד: 308

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `change()`, `useTransactionMutation()`, `useCancelOrder()`, `useTradeHistory()`, `useAuthMe()`, `fetchAll()`, `usePositions()`, `msg()`, `usePendingOrders()`, `useLivePrice()`, `usePreferences()`, `useInvalidateAfterTrade()`

```

import { useState, useEffect } from 'react';
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { apiGet, apiPostJson } from '@lib/api';
import type { BrokerOverview, Position, NotificationItem, Transaction, BrokerOrder, AuthMeResponse }

export { useMutation, useQueryClient };

export const QK = {
  overview: ['broker', 'overview'] as const,
  positions: ['broker', 'positions'] as const,
  notifications: ['broker', 'notifications'] as const,
  transactions: ['broker', 'transactions'] as const,
  history: ['broker', 'history'] as const,
  pendingOrders: ['broker', 'orders', 'pending'] as const,
  me: ['auth', 'me'] as const,
  preferences: ['broker', 'preferences'] as const,
  livePrice: (sym: string) => ['market', 'price', sym.toUpperCase()] as const,
};

/**
 * Custom hook to invalidate React Query cache entities related to trade state.
 * The goal of this hook is to trigger refetching of positions, overview, and history.
 */
export function useInvalidateAfterTrade() {
  const qc = useQueryClient();
  return () => {
    void qc.invalidateQueries({ queryKey: QK.positions });
    void qc.invalidateQueries({ queryKey: QK.overview });
    void qc.invalidateQueries({ queryKey: QK.history });
  };
}

/**
 * Custom hook to fetch general trading account metadata (balance, margin, etc.).
 *
 * @returns React Query query result object
 */
export function useBrokerOverview() {
  return useQuery({
    queryKey: QK.overview,
    queryFn: () => apiGet<BrokerOverview>('/api/broker/overview'),
    refetchInterval: 1000,
    refetchIntervalInBackground: true,
    staleTime: 500,
    retry: 1,
  });
}

```

נתיב קובץ: UI-react/src/hooks/useAuth.tsx | סיומת: .tsx | שורות קוד: 105

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `AuthProvider()`, `useAuth()`, `handler()`

```

import {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useRef,
  useState,
} from 'react';
import type { ReactNode } from 'react';
import { useNavigate } from 'react-router-dom';
import { useQueryClient } from '@tanstack/react-query';
import { apiGet, onUnauthorized } from '@lib/api';
import type { AuthMeResponse } from '@types/api';

type AuthStatus = 'loading' | 'authenticated' | 'unauthenticated';

interface AuthContextValue {
  status: AuthStatus;
  user: AuthMeResponse | null;
  refresh: () => Promise<void>;
  clearUser: () => void;
}

const AuthContext = createContext<AuthContextValue | null>(null);

/**
 * Context Provider component managing user authentication state.
 * Bootstraps initial session, registers unauthorized 401 interceptors, and shares credentials context.
 *
 * @param props children layout elements
 * @returns AuthProvider Context element
 */
export function AuthProvider({ children }: { children: ReactNode }) {
  const [status, setStatus] = useState<AuthStatus>('loading');
  const [user, setUser] = useState<AuthMeResponse | null>(null);
  const navigate = useNavigate();
  const registeredRef = useRef(false);
  const queryClient = useQueryClient();

  /**
   * Resets active credentials state and purges TanStack Query client cache.
   */
  const clearUser = useCallback(() => {
    setUser(null);
  }, []);

```

נתיב קובץ: UI-react/src/hooks/useI18n.tsx | סיומת: .tsx | שורות קוד: 53

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `useI18n()`, `I18nProvider()`

```
import { createContext, useCallback, useContext, useEffect, useMemo, useState } from 'react';
import type { ReactNode } from 'react';
import type { Lang } from '@types/api';
import { getLang, persistLang, translate } from '@lib/i18n';

interface I18nContextValue {
  lang: Lang;
  setLang: (lang: Lang) => void;
  t: (key: string, vars?: Record<string, string | number>) => string;
}

const I18nContext = createContext<I18nContextValue | null>(null);

/**
 * Localization Provider component for translations.
 * Sets the documentElement translation status classes and initializes the active language.
 *
 * @param props children layout elements
 * @returns I18nProvider Context element
 */
export function I18nProvider({ children }: { children: ReactNode }) {
  const [lang, setLangState] = useState<Lang>(() => getLang());

  useEffect(() => {
    persistLang(lang);
    // Legacy CSS hides <body> until this class is set, to avoid flash of untranslated text.
    document.documentElement.classList.add('i18n-ready');
  }, [lang]);

  const setLang = useCallback((next: Lang) => {
    setLangState(next);
  }, []);

  const t = useCallback(
    (key: string, vars?: Record<string, string | number>) => translate(lang, key, vars),
    [lang],
  );

  const value = useMemo<I18nContextValue>(() => ({ lang, setLang, t }), [lang, setLang, t]);

  return <I18nContext.Provider value={value}>{children}</I18nContext.Provider>;
}

/**
 * Custom hook to translate layout text strings or retrieve/change active language.
```

נתיב קובץ: UI-react/src/hooks/useLogout.ts | סיומת: .ts | שורות קוד: 41

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: useLogout()

```
import { useCallback, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { apiPostLogout } from '@lib/api';
import { useAuth } from '@hooks/useAuth';

/**
 * Custom hook providing shared logout logic used by DashboardPage and AccountPage.
 * Clears server session, React Query cache, localStorage settings,
 * and resets DOM to defaults before navigating to /login.
 *
 * @returns object containing logout trigger function and loggingOut state flag
 */
export function useLogout() {
  const { clearUser } = useAuth();
  const navigate = useNavigate();
  const [loggingOut, setLoggingOut] = useState(false);

  /**
   * Performs the programmatic logout sequence.
   * Calls API logout endpoints, purges settings, cleans DOM elements, resets Auth status, and redirects.
   */
  const logout = useCallback(async () => {
    setLoggingOut(true);
    try {
      await apiPostLogout();
    } catch {
      // ignore server errors – always clear local state
    } finally {
      localStorage.removeItem('theme');
      localStorage.removeItem('broker-ui-lang');
      document.documentElement.setAttribute('data-theme', 'dark');
      document.documentElement.lang = 'en';
      document.documentElement.dir = 'ltr';
      clearUser();
      setLoggingOut(false);
      navigate('/login', { replace: true });
    }
  }, [clearUser, navigate]);

  return { logout, loggingOut };
}
```

נתיב קובץ: UI-react/src/hooks/useNotificationsWs.ts | סיומת: .ts | שורות קוד: 47

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: useNotificationsWs()

```
import { useEffect, useRef } from 'react';
import { Client } from '@stomp/stompjs';
import SockJS from 'sockjs-client';
import { useQueryClient } from '@tanstack/react-query';
import { QK } from './useApi';
import { IS_MOCK } from '@lib/api';
import type { NotificationItem } from '@types/api';

/**
 * Custom hook to establish and maintain a WebSockets connection (SockJS/STOMP) for notifications.
 * Automatically pushes new message entities into the React Query cache or triggers invalidations.
 *
 * @param enabled flag to initialize WebSocket connect, defaults to true
 */
export function useNotificationsWs(enabled = true) {
  const qc = useQueryClient();
  const clientRef = useRef<Client | null>(null);

  useEffect(() => {
    if (!enabled || IS_MOCK) return;

    const client = new Client({
      websocketFactory: () => new SockJS('/ws'),
      reconnectDelay: 5000,
      onConnect: () => {
        client.subscribe('/user/queue/notifications', (msg) => {
          try {
            const notification = JSON.parse(msg.body) as NotificationItem;
            qc.setQueryData<NotificationItem[]>(QK.notifications, (old = []) => [
              notification,
              ...old,
            ]);
          } catch {
            qc.invalidateQueries({ queryKey: QK.notifications });
          }
        });
      },
    });

    client.activate();
    clientRef.current = client;

    return () => {
      client.deactivate();
    };
  }, []);
}
```

נתיב קובץ: UI-react/src/hooks/useToast.tsx | סימנת: .tsx | שורות קוד: 105

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `useToast()`, `ToastProvider()`

```
import { createContext, useCallback, useContext, useMemo, useRef, useState } from 'react';
import type { ReactNode } from 'react';
import type { ToastVariant } from '@types/api';

interface ToastItem {
  id: number;
  message: string;
  variant: ToastVariant;
  leaving: boolean;
}

interface ToastOptions {
  variant?: ToastVariant;
  duration?: number;
}

interface ToastContextValue {
  show: (message: string, options?: ToastOptions) => void;
}

const ToastContext = createContext<ToastContextValue | null>(null);

const DEFAULT_DURATION = 4200;
const FADE_MS = 320;

/**
 * Context Provider component managing popup alert messages.
 * Mounts the dynamic fixed toast-container overlay in the viewport layout.
 *
 * @param props children layout elements
 * @returns ToastProvider Context element
 */
export function ToastProvider({ children }: { children: ReactNode }) {
  const [toasts, setToasts] = useState<ToastItem[]>([]);
  const idCounter = useRef(0);
  const timersRef = useRef(new Map<number, ReturnType<typeof setTimeout>>());

  /**
   * Internal helper to trigger transition animations and remove a toast by ID.
   *
   * @param id targeted toast identifier
   */
  const remove = useCallback((id: number) => {
    setToasts((prev) => prev.map((t) => (t.id === id ? { ...t, leaving: true } : t)));
    setTimeout(() => {
```

נתיב קובץ: UI-react/src/lib/api.ts | סיומת: .ts | שורות קוד: 996

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `marginReturned()`, `apiPostJson()`, `parseOverviewValue()`, `getLivePrice()`, `apiPostLogout()`, `xsrftokenFromCookie()`, `postWithCsrfRetry()`, `xsrferHeader()`, `processMockSltP()`, `processMockPendingOrders()`, `doReq()`, `getContractSize()`

```

// Typed wrapper around fetch – preserves CSRF + cookie auth from the legacy api.js.

// Toggle to control whether we run in offline Mock Mode (recommended for GitHub Pages portfolio)
export const IS MOCK = localStorage.getItem('force_mock') === 'true' ||
    (!window.location.hostname.includes('localhost') &&
    !window.location.hostname.includes('127.0.0.1'));

// Fired whenever the server returns 401. Components can listen and redirect.
export const onUnauthorized: Array<() => void> = [];

/**
 * Extracts the XSRF token value from the cookies mapping.
 *
 * @returns token string or empty string if not found
 */
export function xsrfTokenFromCookie(): string {
    const m = document.cookie.match(/(?:^|;|s*)XSRF-TOKEN=([^\s]*)/);
    return m ? decodeURIComponent(m[1]) : '';
}

/**
 * Ensures a CSRF validation cookie has been initialized by calling the csrf endpoint.
 *
 * @throws Error if response is not ok
 */
export async function ensureCsrfCookie(): Promise<void> {
    if (IS MOCK) return;
    const res = await fetch('/api/auth/csrf', {
        method: 'GET',
        credentials: 'include',
        headers: { Accept: 'application/json' },
    });
    if (!res.ok) {
        const text = await res.text().catch(() => '');
        throw new Error(`CSRF init failed: ${res.status} ${text}`);
    }
    await res.json();
}

/**
 * Builds the X-XSRF-TOKEN header entry object.
 *
 * @returns headers record mapping object
 */
async function xsrfHeader(): Promise<Record<string, string>> {

```

נתיב קובץ: UI-react/src/lib/i18n.ts | סיומת: .ts | שורות קוד: 84

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `getLang()`, `translate()`, `wrapHeSubstitution()`, `persistLang()`, `isAllowed()`

```

import MESSAGES from '@locales';
import type { Lang } from '@types/api';

export const LANG_KEY = 'broker-ui-lang';

const ALLOWED: ReadonlySet<Lang> = new Set<Lang>(['en', 'ru', 'he']);

type LocaleDict = Record<string, string>;
type LocalesMap = Record<Lang, LocaleDict>;

const messages = MESSAGES as unknown as LocalesMap;

/**
 * Checks if language code is within ALLOWED language list.
 *
 * @param code language code identifier
 * @returns true if allowed, false otherwise
 */
function isAllowed(code: string): code is Lang {
  return ALLOWED.has(code as Lang);
}

/**
 * Reads local storage preference to determine language code.
 *
 * @returns configured active Lang code
 */
export function getLang(): Lang {
  try {
    const v = localStorage.getItem(LANG_KEY) ?? 'en';
    return isAllowed(v) ? v : 'en';
  } catch {
    return 'en';
  }
}

/**
 * Saves language preference choice to local storage and updates document HTML directions/attributes
 *
 * @param lang active target Lang
 */
export function persistLang(lang: Lang): void {
  try {
    localStorage.setItem(LANG_KEY, lang);
  } catch {

```

נתיב קובץ: UI-react/src/lib/userProfile.ts | סיומת: .ts | שורות קוד: 58

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `getUserProfile()`, `saveUserProfile()`

```
export const USER_PROFILE_KEY = 'broker-ui-user-profile';

export interface UserProfile {
  uid?: string;
  firstName?: string;
  lastName?: string;
  idDoc?: string;
  email?: string;
  phone?: string;
  currency?: string;
  leverage?: string;
  updatedAt?: number;
  [extra: string]: unknown;
}

/**
 * Retrieves the cached client-side user profile information from localStorage.
 *
 * @returns UserProfile object or null if none is saved/parse error occurs
 */
export function getUserProfile(): UserProfile | null {
  try {
    const raw = localStorage.getItem(USER_PROFILE_KEY);
    if (!raw) return null;
    const p = JSON.parse(raw) as unknown;
    return typeof p === 'object' && p !== null ? (p as UserProfile) : null;
  } catch {
    return null;
  }
}

/**
 * Saves profile data to local state (for offline support) and guarantees a unique UID.
 *
 * @param data partial profile fields to merge
 * @returns updated full UserProfile object
 */
export function saveUserProfile(data: Partial<UserProfile>): UserProfile {
  const prev = getUserProfile() ?? {};
  const uid =
    prev.uid ??
    `BRK-${Date.now().toString(36).toUpperCase()}-${Math.random()
      .toString(36)
      .slice(2, 6)
      .toUpperCase()}`;
}
```

נתיב קובץ: UI-react/src/locales.json | סיומת: .json | שורות קוד: 1133

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
{
  "en": {
    "nav.positions": "Positions",
    "nav.charts": "Charts",
    "nav.finance": "Finance",
    "nav.history": "History",
    "nav.account": "Account",
    "nav.settings": "Settings",
    "nav.dashboard": "Dashboard",
    "nav.analyzer": "AI Analyzer",
    "common.back": "Back",
    "common.or": "OR",
    "common.close": "Close",
    "common.edit": "Edit",
    "common.manage": "Manage",
    "common.filter": "Filter",
    "common.symbol": "Symbol:",
    "common.from": "From:",
    "common.to": "To:",
    "common.search": "Search...",
    "common.loading": "Loading...",
    "titles.landing": "Home - Broker App",
    "titles.login": "Login - Broker App",
    "titles.register": "Register - Broker App",
    "titles.demoRegister": "Demo Registration - Broker App",
    "titles.dashboard": "Dashboard - Broker App",
    "titles.positions": "Positions - Broker App",
    "titles.charts": "Charts - Broker App",
    "titles.trading": "Trading - Broker App",
    "titles.finance": "Finance - Broker App",
    "titles.history": "Trade History - Broker App",
    "titles.account": "Account - Broker App",
    "titles.settings": "Settings - Broker App",
    "titles.broker": "Admin Dashboard - Broker App",
    "titles.admin": "Admin - Broker App",
    "landing.hero": "Trade the World's Markets with One Platform",
    "landing.openAccount": "Open Account",
    "landing.demoAccount": "Demo Account",
    "landing.logIn": "Log In",
    "landing.signUp": "Sign Up",
    "landing.assets": "Trading Assets",
    "landing.spread": "Minimal Spread",
    "landing.leverage": "Max Leverage",
    "login.heading": "Log In",
    "login.sub": "Access your trading account",
```

נתיב קובץ: UI-react/src/main.tsx | סיומת: .tsx | שורות קוד: 24

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import { App } from './App';
import './styles/main.css';

// Restore theme before first paint to avoid flicker.
const savedTheme = localStorage.getItem('theme');
if (savedTheme) {
  document.documentElement.setAttribute('data-theme', savedTheme);
}

/**
 * Initial bootstrapper file for the React client application.
 * Restores the persisted styling theme from localStorage before render,
 * verifies DOM root container presence, and mounts the React application under StrictMode.
 */
const rootEl = document.getElementById('root');
if (!rootEl) throw new Error('Root element #root not found');

createRoot(rootEl).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

נתיב קובץ: UI-react/src/pages/AccountPage.tsx | סיומת: .tsx | שורות קוד: 163

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `AccountPage()`, `handleLogout()`, `handleEdit()`, `displayOrDash()`

```

import { useEffect } from 'react';
import { getUserProfile } from '@lib/userProfile';
import { useI18n } from '@hooks/useI18n';
import { useToast } from '@hooks/useToast';
import { BackPageHeader } from '@components/BackPageHeader';
import { Skeleton } from '@components/Skeleton';
import { useBrokerOverview, useAuthMe } from '@hooks/useApi';
import { useLogout } from '@hooks/useLogout';

/**
 * Formatting utility to return a dash symbol if string is null or empty.
 *
 * @param value raw input value
 * @returns formatted string or dash
 */
function displayOrDash(value: unknown): string {
  if (value == null) return '-';
  const s = String(value).trim();
  return s || '-';
}

/**
 * Account page displaying currently logged-in user profile attributes,
 * security credentials, and registration files.
 * The goal of this page is to view user personal credentials and log out.
 *
 * @returns Account view page element
 */
export function AccountPage() {
  const { t } = useI18n();
  const toast = useToast();
  const { logout, loggingOut } = useLogout();

  const { data: me, isLoading: meLoading } = useAuthMe();
  const { data: overview, isLoading: overviewLoading } = useBrokerOverview();

  const isLoading = meLoading || overviewLoading;

  useEffect(() => {
    document.title = t('titles.account');
  }, [t]);

  const profile = getUserProfile();

  const displayName = me?.displayName || (profile ? [profile.firstName, profile.lastName].filter(Boolean)

```

נתיב קובץ: UI-react/src/pages/AnalyzerPage.tsx | סיומת: .tsx | שורות קוד: 461

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `AnalyzerPage()`, `fetchOverview()`, `handleAssetSelect()`, `fetchAnalysis()`, `handleTradeNow()`, `getRecBadgeClass()`

```

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useI18n } from '@/hooks/useI18n';
import { BackPageHeader } from '@/components/BackPageHeader';

interface AnalysisResult {
  symbol: string;
  recommendation: string;
  recommendationHe: string;
  score: number;
  confidence: number;
  color: string;
  price: number;
  indicators: {
    rsi: { value: number; status: string; statusHe: string } | null;
    macd: { macdLine: number; signalLine: number; hist: number; status: string; statusHe: string };
    sma: { sma20: number; sma50: number; status: string; statusHe: string } | null;
    bb: { upper: number; middle: number; lower: number; status: string; statusHe: string } | null;
  };
  explanationsEn: string[];
  explanationsHe: string[];
  summary: string;
  summaryHe: string;
}

interface OverviewAsset {
  symbol: string;
  category: string;
  price: number;
  recommendation: string;
  recommendationHe: string;
  confidence: number;
  color: string;
  rsi: number;
  macdStatus: string;
  macdStatusHe: string;
}

export function AnalyzerPage() {
  const { t, lang } = useI18n();
  const navigate = useNavigate();

  const [assets, setAssets] = useState<OverviewAsset[]>([]);
  const [selectedSymbol, setSelectedSymbol] = useState<string>('BTCUSD');
  const [selectedCategory, setSelectedCategory] = useState<string>('crypto');

```

נתיב קובץ: UI-react/src/pages/ChartsPage.tsx | סיומת: .tsx | שורות קוד: 1128

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `symbolAnnualVol()`, `positions()`, `barsPerYear()`, `stopLive()`, `periodKey()`, `placeOrder()`, `updateBar()`, `defaultPrice()`, `handleCancelOrder()`, `updateBidAsk()`, `isLightTheme()`, `fetchBybitKlines()`

```

import { useEffect, useRef, useState, useCallback } from 'react';
import { useSearchParams } from 'react-router-dom';
import {
  createChart, ColorType, CrosshairMode,
  CandlestickSeries, HistogramSeries,
  type IChartApi, type ISeriesApi, type UTCTimestamp,
} from 'lightweight-charts';
import { apiPostJson, getContractSize } from '@lib/api';
import { useI18n } from '@hooks/useI18n';
import { useToast } from '@hooks/useToast';
import { BackPageHeader } from '@components/BackPageHeader';
import { LoanOfferModal, type LoanOfferDetails } from '@components/LoanOfferModal';
import { usePositions, useBrokerOverview, useInvalidateAfterTrade, useLivePrices, usePendingOrders,

import type { PlaceOrderResponse, ClosePositionResponse } from '@types/api';

// — Constants —————

const BYBIT_KLINES_URL = 'https://api.bybit.com/v5/market/kline';
const BYBIT_WS = 'wss://stream.bybit.com/v5/public/linear';
const STOCK_POLL_MS = 5000;
const FETCH_TIMEOUT_MS = 10_000;

type Category = 'forex' | 'metals' | 'crypto';
type OrderType = 'MARKET' | 'LIMIT';
type Interval = '1m' | '5m' | '1h' | '4h' | '1d' | '1w' | '1M';

interface Instrument { id: string; title: string; decimals: number; source?: string }
interface Candle { time: UTCTimestamp; open: number; high: number; low: number; close: number; volum

const CATEGORY_CONFIG: Record<Category, { instruments: Instrument[]; source: string }> = {
  forex: {
    source: 'yahoo',
    instruments: [
      { id: 'EURUSD', title: 'EURUSD', decimals: 5 },
      { id: 'GBPUSD', title: 'GBPUSD', decimals: 5 },
      { id: 'USDCAD', title: 'USDCAD', decimals: 5 },
      { id: 'EURNOK', title: 'EURNOK', decimals: 5 },
      { id: 'GBPJPY', title: 'GBPJPY', decimals: 3 },
      { id: 'USDJPY', title: 'USDJPY', decimals: 3 },
      { id: 'NZDUSD', title: 'NZDUSD', decimals: 5 },
      { id: 'CADJPY', title: 'CADJPY', decimals: 3 },
    ],
  },
  metals: {

```

נתיב קובץ: UI-react/src/pages/DashboardPage.tsx | סיומת: .tsx | שורות קוד: 709

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `fmtPnl()`, `isToday()`, `DashboardPage()`, `fmtPrice()`, `fmtMoney()`

```

import { useEffect } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useI18n } from '@/hooks/useI18n';
import { useToast } from '@/hooks/useToast';
import { useLogout } from '@/hooks/useLogout';
import { DashboardHeader } from '@/components/DashboardHeader';
import { SkeletonCard, SkeletonRow } from '@/components/Skeleton';
import { useBrokerOverview, usePositions, useNotifications, useLivePrices, useTradeHistory } from '@
import { getContractSize } from '@/lib/api';

/**
 * Formatting utility to render numeric values as currency localized strings.
 *
 * @param value raw numeric input value
 * @param currency target currency code, defaults to USD
 * @returns formatted currency string
 */
function fmtMoney(value: unknown, currency = 'USD'): string {
  const n = Number(value ?? 0);
  return new Intl.NumberFormat(undefined, {
    style: 'currency',
    currency,
    minimumFractionDigits: 2,
    maximumFractionDigits: 2,
  }).format(n);
}

function fmtPnl(n: unknown): string {
  const v = Number(n ?? 0);
  if (!Number.isFinite(v)) return '-';
  const abs = Math.abs(v);
  if (abs === 0) return '0.00';
  if (abs >= 0.01) return v.toLocaleString(undefined, { minimumFractionDigits: 2, maximumFractionDig
  return v.toLocaleString(undefined, { minimumFractionDigits: 2, maximumFractionDigits: 8 });
}

/**
 * Formatting utility to render numeric values as price quotes with up to 5 decimals.
 *
 * @param n raw numeric price quote input
 * @returns formatted price quote string
 */
function fmtPrice(n: unknown): string {
  if (n == null) return '-';
  const v = Number(n);

```

נתיב קובץ: UI-react/src/pages/DemoRegisterPage.tsx | סיומת: .tsx | שורות קוד: 283

תפקיד הקובץ במערכת: רכיב UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `DemoRegisterPage()`, `onSubmit()`

```
import { useEffect } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { apiPostJson, apiPostFormUrlEncoded } from '@lib/api';
import { useI18n } from '@hooks/useI18n';
import { useToast } from '@hooks/useToast';
import { saveUserProfile } from '@lib/userProfile';
import { AuthHeader } from '@components/AuthHeader';
import { useAuth } from '@hooks/useAuth';

const CURRENCIES = ['USD', 'EUR', 'GBP'] as const;
const LEVERAGES = ['1:100', '1:50', '1:500'] as const;

const schema = z.object({
  firstName: z.string().optional(),
  lastName: z.string().optional(),
  email: z.string().email('Invalid email'),
  password: z.string().min(8, 'Password must be at least 8 characters'),
  currency: z.enum(['USD', 'EUR', 'GBP']),
  leverage: z.enum(['1:100', '1:50', '1:500']),
  terms: z.literal(true, { message: 'You must accept terms' }),
  risk: z.literal(true, { message: 'You must accept risk disclosure' }),
});
type FormData = z.infer<typeof schema>;

/**
 * Registration page for creating DEMO trading accounts.
 * Validates fields via Zod, sets up local storage profile values, registers user credentials on the
 * and signs the user in.
 * The goal of this page is to establish a demo account configuration for paper trading simulation.
 *
 * @returns Demo registration form page layout
 */
export function DemoRegisterPage() {
  const { t } = useI18n();
  const toast = useToast();
  const navigate = useNavigate();
  const { refresh } = useAuth();

  useEffect(() => {
    document.title = t('titles.demoRegister');
  }, [t]);
```

נתיב קובץ: UI-react/src/pages/FinancePage.tsx | סיומת: .tsx | שורות קוד: 360

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `fmtMoney()`, `selectPreset()`, `handleConfirm()`, `TxModal()`, `fmtTime()`

`FinancePage()`

```

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useI18n } from '@/hooks/useI18n';
import { useToast } from '@/hooks/useToast';
import { BackPageHeader } from '@/components/BackPageHeader';
import { SkeletonRow } from '@/components/Skeleton';
import { useBrokerOverview, useTransactions, useTransactionMutation } from '@/hooks/useApi';

type TxType = 'DEPOSIT' | 'WITHDRAWAL';

const PRESETS = [500, 1000, 5000, 10000];

function fmtMoney(n: unknown, currency = 'USD'): string {
  return new Intl.NumberFormat(undefined, {
    style: 'currency',
    currency,
    maximumFractionDigits: 2,
  }).format(Number(n ?? 0));
}

function fmtTime(iso: string | undefined): string {
  if (!iso) return '-';
  const d = new Date(iso);
  return Number.isNaN(d.getTime()) ? '-' : d.toLocaleString();
}

interface TxModalProps {
  type: TxType;
  currency: string;
  onClose: () => void;
  onConfirm: (amount: number, method: string) => Promise<void>;
  t: (key: string) => string;
}

function TxModal({ type, onClose, onConfirm, t }: TxModalProps) {
  const [amount, setAmount] = useState('');
  const [method, setMethod] = useState('card');
  const [preset, setPreset] = useState<number | null>(null);
  const [busy, setBusy] = useState(false);
  const isDeposit = type === 'DEPOSIT';

  async function handleConfirm() {
    const val = Number(amount);
    if (!Number.isFinite(val) || val <= 0) return;
    setBusy(true);
  }

```

נתיב קובץ: UI-react/src/pages/HistoryPage.tsx | סיומת: .tsx | שורות קוד: 365

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `exportCsv()`, `fmtPl()`, `fmtPrice()`, `HistoryPage()`, `fmtQty()`, `ts()`, `pairOrders()`, `fmtTime()`, `applyFilters()`, `handleReset()`

```

import { useEffect, useRef, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useI18n } from '@/hooks/useI18n';
import { useToast } from '@/hooks/useToast';
import { BackPageHeader } from '@/components/BackPageHeader';
import { SkeletonRow } from '@/components/Skeleton';
import { useBrokerOverview, useTradeHistory } from '@/hooks/useApi';
import type { BrokerOrder } from '@/types/api';

// — Types —————

interface PairedTrade {
  id: number;
  symbolCode: string;
  quantity: string;
  openTime?: string;
  openPrice?: string;
  closeTime?: string;
  closePrice?: string;
  stopLoss?: string;
  takeProfit?: string;
  realizedPnl?: string;
  commission?: string;
}

// — Pairing logic —————
// Match BUY → SELL chronologically per symbol (FIFO).
// Returns only completed pairs (SELL side).

function pairOrders(orders: BrokerOrder[]): PairedTrade[] {
  // Sort all orders by date ascending
  const sorted = [...orders].sort(
    (a, b) => ts(a.filledAt || a.createdAt) - ts(b.filledAt || b.createdAt)
  );

  const closing = sorted.filter(
    (o) => o.realizedPnl != null || o.openPrice != null
  );
  const opening = sorted.filter(
    (o) => o.realizedPnl == null && o.openPrice == null
  );

  const usedOpeningIds = new Set<number>();
  const pairs: PairedTrade[] = [];

```

נתיב קובץ: UI-react/src/pages/LandingPage.tsx | סימנת: .tsx | שורות קוד: 186

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: LandingPage()

```
import { useEffect } from 'react';
import { Link } from 'react-router-dom';
import { apiGet } from '@lib/api';
import { useI18n } from '@hooks/useI18n';
import { useToast } from '@hooks/useToast';
import { AuthHeader } from '@components/AuthHeader';

/**
 * Welcome marketing landing page for the TradeAge Broker Platform.
 * Displays call to action buttons to direct users to register, log in, or try the demo mode.
 * The goal of this page is to serve as the initial entrance interface of the website application.
 *
 * @returns Landing page layout
 */
export function LandingPage() {
  const { t } = useI18n();
  const toast = useToast();

  useEffect(() => {
    document.title = t('titles.landing');
    // Dev smoke check: ping backend so we know API is reachable.
    apiGet('/api/health')
      .then(() => toast.show('API: OK', { variant: 'success', duration: 1400 }))
      .catch(() => {
        /* backend optional during static UI work */
      });
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  return (
    <>
      <AuthHeader>
        <div className="flex-gap">
          <Link to="/login" className="btn btn-outline">
            {t('landing.logIn')}
          </Link>
          <Link to="/register" className="btn btn-primary">
            {t('landing.signUp')}
          </Link>
        </div>
      </AuthHeader>

      <div className="container text-center mt-20" style={{ padding: '60px 20px 20px' }}>
        <h1 style={{ fontSize: '2.5rem', marginBottom: 10 }}>{t('landing.hero')}</h1>
      </div>
    </>
  );
}
```

נתיב קובץ: UI-react/src/pages/LoginPage.tsx | סיומת: .tsx | שורות קוד: 198

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `LoginPage()`, `handleGoogle()`, `handleForgot()`, `from()`, `onSubmit()`

```

import { useEffect } from 'react';
import { Link, useNavigate, useLocation } from 'react-router-dom';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { apiPostFormUrlEncoded } from '@lib/api';
import { useI18n } from '@hooks/useI18n';
import { useToast } from '@hooks/useToast';
import { useAuth } from '@hooks/useAuth';
import { AuthHeader } from '@components/AuthHeader';

const schema = z.object({
  email: z.string().email('Invalid email'),
  password: z.string().min(1, 'Password is required'),
});
type FormData = z.infer<typeof schema>;

/**
 * Login Page that handles credentials login submissions to the backend database.
 * The goal of this page is to authenticate the user and redirect them to their target dashboard path
 *
 * @returns Login page layout
 */
export function LoginPage() {
  const { t } = useI18n();
  const toast = useToast();
  const navigate = useNavigate();
  const location = useLocation();
  const { refresh, status } = useAuth();

  const from = (location.state as { from?: { pathname?: string } } | null)?.from?.pathname ?? '/dash

  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm<FormData>({
    resolver: zodResolver(schema),
  });

  useEffect(() => {
    document.title = t('titles.login');
  }, [t]);

  useEffect(() => {

```

נתיב קובץ: UI-react/src/pages/PlaceholderPage.tsx | סיומת: .tsx | שורות קוד: 27

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: PlaceholderPage()

```
import { Link } from 'react-router-dom';

interface Props {
  title: string;
}

/**
 * Simple fallback page representing non-migrated endpoints or 404 targets.
 *
 * @param props title header key string
 * @returns Placeholder fallback component
 */
export function PlaceholderPage({ title }: Props) {
  return (
    <div className="container" style={{ maxWidth: 600, marginTop: 80 }}>
      <div className="card text-center">
        <h2>{title}</h2>
        <p style={{ color: 'var(--text-secondary)', marginTop: 12 }}>
          This page has not been migrated to React yet.
        </p>
        <Link to="/login" className="btn btn-primary" style={{ marginTop: 20 }}>
          Go to Login
        </Link>
      </div>
    </div>
  );
}
```

נתיב קובץ: UI-react/src/pages/PositionsPage.tsx | סיומת: .tsx | שורות קוד: 493

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `tsMs()`, `PositionsPage()`, `pad()`, `data()`, `fmtPrice()`, `fmtQty()`, `fmtPnl()`, `pairOrders()`, `fmtTime()`, `handleCancelOrder()`, `ConfirmModal()`, `closePosition()`

```

import { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { apiPostJson, getContractSize } from '@lib/api';
import { useI18n } from '@hooks/useI18n';
import { useToast } from '@hooks/useToast';
import { BackPageHeader } from '@components/BackPageHeader';
import { SkeletonRow } from '@components/Skeleton';
import { usePositions, useLivePrices, useInvalidateAfterTrade, useTradeHistory, usePendingOrders, use }
import type { Position, BrokerOrder } from '@types/api';

interface PairedTrade {
  id: number;
  symbolCode: string;
  quantity: string;
  openTime?: string;
  openPrice?: string;
  closeTime?: string;
  closePrice?: string;
  stopLoss?: string;
  takeProfit?: string;
  realizedPnl?: string;
}

function tsMs(iso?: string) { return iso ? new Date(iso).getTime() : 0; }

function fmtPnl(n: unknown): string {
  const v = Number(n ?? 0);
  if (!Number.isFinite(v)) return '-';
  const abs = Math.abs(v);
  if (abs === 0) return '0.00';
  if (abs >= 0.01) return v.toLocaleString(undefined, { minimumFractionDigits: 2, maximumFractionDig
  return v.toLocaleString(undefined, { minimumFractionDigits: 2, maximumFractionDigits: 8 });
}

function pairOrders(orders: BrokerOrder[]): PairedTrade[] {
  const sorted = [...orders].sort(
    (a, b) => tsMs(a.filledAt || a.createdAt) - tsMs(b.filledAt || b.createdAt)
  );

  const closing = sorted.filter(
    (o) => o.realizedPnl != null || o.openPrice != null
  );
  const opening = sorted.filter(
    (o) => o.realizedPnl == null && o.openPrice == null
  );

```

נתיב קובץ: UI-react/src/pages/RegisterPage.tsx | סימנת: .tsx | שורות קוד: 268

תפקיד הקובץ במערכת: רכיב UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `RegisterPage()`, `onSubmit()`

```
import { useEffect } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';
import { apiPostJson } from '@lib/api';
import { useI18n } from '@hooks/useI18n';
import { useToast } from '@hooks/useToast';
import { saveUserProfile } from '@lib/userProfile';
import { AuthHeader } from '@components/AuthHeader';

const CURRENCIES = ['USD', 'EUR', 'GBP'] as const;
const LEVERAGES = ['1:100', '1:50', '1:500'] as const;

const schema = z.object({
  firstName: z.string().optional(),
  lastName: z.string().optional(),
  idDoc: z.string().optional(),
  email: z.string().email('Invalid email'),
  phone: z.string().optional(),
  password: z.string().min(8, 'Password must be at least 8 characters'),
  currency: z.enum(['USD', 'EUR', 'GBP']),
  leverage: z.enum(['1:100', '1:50', '1:500']),
  terms: z.literal(true, { message: 'You must accept terms' }),
  risk: z.literal(true, { message: 'You must accept risk disclosure' }),
});
type FormData = z.infer<typeof schema>;

/**
 * Registration Page for registering new REAL broker client accounts.
 * Collects personal identification files, contact information, passwords, and currency settings.
 * The goal of this page is to securely enroll a real user account in the broker database.
 *
 * @returns Real account registration page layout
 */
export function RegisterPage() {
  const { t } = useI18n();
  const toast = useToast();
  const navigate = useNavigate();

  const {
    register,
    handleSubmit,
    formState: { errors, isSubmitting },
  } = useForm<FormData>({
```

נתיב קובץ: UI-react/src/pages/SettingsPage.tsx | סיומת: .tsx | שורות קוד: 271

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `SettingsPage()`, `handleThemeChange()`, `ToggleRow()`, `handleToggle()`, `applyTheme()`, `handleChangePassword()`, `handleLangChange()`

```

import { useEffect, useState } from 'react';
import { useI18n } from '@/hooks/useI18n';
import { useToast } from '@/hooks/useToast';
import { BackPageHeader } from '@/components/BackPageHeader';
import { usePreferences, useSavePreferences } from '@/hooks/useApi';
import { Skeleton } from '@/components/Skeleton';
import type { Lang } from '@/types/api';

type Theme = 'dark' | 'light' | 'system';

/**
 * Internal helper to set theme styles to document elements and persist selection to local storage.
 *
 * @param theme style parameter theme name
 */
function applyTheme(theme: Theme) {
  const root = document.documentElement;
  if (theme === 'system') {
    const prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
    root.setAttribute('data-theme', prefersDark ? 'dark' : 'light');
  } else {
    root.setAttribute('data-theme', theme);
  }
  localStorage.setItem('theme', theme);
}

interface ToggleRowProps {
  label: string;
  description: string;
  checked: boolean;
  onChange: (v: boolean) => void;
  disabled?: boolean;
  borderBottom?: boolean;
}

/**
 * Helper row toggle layout component rendering checkbox control switch.
 *
 * @param props styling/functional parameters of toggle
 * @returns ToggleRow container element
 */
function ToggleRow({ label, description, checked, onChange, disabled, borderBottom = true }: ToggleRowProps) {
  return (
    <div
      className="flex-between"

```

נתיב קובץ: UI-react/src/types/api.ts | סיומת: .ts | שורות קוד: 129

תפקיד הקובץ במערכת: רכיב UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
// Backend API response types – keep in sync with Spring Boot DTOs.

export type Lang = 'en' | 'ru' | 'he';

export type ToastVariant = 'info' | 'success' | 'warning' | 'error';

export interface ApiError {
  error?: string;
  message?: string;
  status?: number;
}

export interface User {
  id: number;
  email: string;
  displayName?: string;
  role?: string;
  banned?: boolean;
  createdAt?: string;
}

export interface AuthMeResponse {
  id: number;
  email: string;
  displayName: string;
  role: string;
  createdAt?: string;
  banned?: boolean;
}

export interface BrokerOverview {
  tradingAccountId: number;
  accountType: string;
  currency: string;
  leverage: number;
  balance: number | string;
  equity: number | string;
  marginUsed?: number | string;
  freeMargin?: number | string;
  borrowedBalance?: number | string;
  creditLimit?: number | string;
  marginLevelPct?: number | string | null;
  interestAccruedTotal?: number | string;
  commissionPaidTotal?: number | string;
  dailyInterestRate?: number | string;
}
```

נתיב קובץ: UI-react/src/vite-env.d.ts | סיומת: .ts | שורות קוד: 1

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
/// <reference types="vite/client" />
```

נתיב קובץ: UI-react/tsconfig.json | סימנת: .json | שורות קוד: 30

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,

    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "moduleDetection": "force",
    "noEmit": true,
    "jsx": "react-jsx",

    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,

    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"],
      "@locales": ["src/locales.json"]
    }
  },
  "include": ["src"],
  "references": [{ "path": "./tsconfig.node.json" }]
}
```

נתיב קובץ: UI-react/tsconfig.node.json | סיומת: .json | שורות קוד: 11

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
{
  "compilerOptions": {
    "composite": true,
    "skipLibCheck": true,
    "module": "ESNext",
    "moduleResolution": "bundler",
    "allowSyntheticDefaultImports": true,
    "strict": true
  },
  "include": ["vite.config.ts"]
}
```

נתיב קובץ: UI-react/vite.config.ts | סיומת: .ts | שורות קוד: 30

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { resolve, dirname } from 'path';
import { fileURLToPath } from 'url';

const __dirname = dirname(fileURLToPath(import.meta.url));

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      '@': resolve(__dirname, 'src'),
      '@locales': resolve(__dirname, 'src/locales.json'),
    },
  },
  server: {
    port: 3001,
    host: '127.0.0.1',
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
      },
    },
  },
  build: {
    outDir: 'dist',
    emptyOutDir: true,
  },
});
```

נתיב קובץ: UI_alfa_Comprehensive_System_Guide.html | סיומת: .html | שורות קוד: 5972

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `getUserProfile()`, `saveUserProfile()`, `useNotificationsWs()`, `fmtMoney()`, `ToastProvider()`, `BackPageHeader()`, `NNAdvisorPanel()`, `displayOrDash()`, `askQuestion()`, `fmtPrice()`, `xsrHeader()`, `SkeletonCard()`

```

<!DOCTYPE html>
<html lang="he" dir="rtl">
<head>
  <meta charset="UTF-8">
  <title>מלא מחלקות ומילון מנגנונים, ארכיטקטורה, UI_alfa Brokerage Platform</title>
  <style>
    @page {
      size: A4;
      margin: 18mm 15mm 18mm 15mm;
      @bottom-center {
        content: "עמוד " counter(page);
        font-family: sans-serif;
        font-size: 9pt;
        color: #64748b;
      }
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      color: #0f172a;
      line-height: 1.55;
      background-color: #ffffff;
      margin: 0;
      padding: 0;
      font-size: 9.5pt;
    }

    .cover-page {
      page-break-after: always;
      text-align: center;
      border: 3px solid #0284c7;
      padding: 40px 25px;
      border-radius: 12px;
      background-color: #f0f9ff;
      margin-bottom: 30px;
    }

    .cover-title {
      font-size: 24pt;
      font-weight: 800;
      color: #0f172a;
      margin-top: 15px;
      margin-bottom: 10px;
    }
  </style>

```

נתיב קובץ: demo_matching_cases.js | סיומת: .js | שורות קוד: 245

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `run()`, `post()`, `askQuestion()`, `loginAndGetHeaders()`, `get()`, `resetDatabase()`

```

const http = require('http');
const readline = require('readline');
const { execSync } = require('child_process');

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

function askQuestion(query) {
  return new Promise((resolve) => rl.question(query, resolve));
}

function post(url, data, headers = {}) {
  return new Promise((resolve, reject) => {
    const u = new URL(url);
    const isJson = typeof data === 'object';
    const body = isJson ? JSON.stringify(data) : data;
    const options = {
      hostname: u.hostname,
      port: u.port || 80,
      path: u.pathname + u.search,
      method: 'POST',
      headers: {
        ...headers,
        'Content-Type': isJson ? 'application/json' : 'application/x-www-form-urlencoded',
        'Content-Length': Buffer.byteLength(body)
      }
    };

    const req = http.request(options, (res) => {
      let chunk = '';
      res.on('data', (d) => { chunk += d.toString(); });
      res.on('end', () => {
        resolve({
          status: res.statusCode,
          headers: res.headers,
          body: chunk
        });
      });
    });

    req.on('error', reject);
    req.write(body);
    req.end();
  });
}

```

נתיב קובץ: erd_document.html | סיומת: .html | שורות קוד: 332

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Entity Relationship Diagram - Broker Platform</title>
  <script src="https://cdn.jsdelivr.net/npm/mermaid/dist/mermaid.min.js"></script>
  <style>
    @page {
      size: A4 portrait;
      margin: 15mm;
    }
    body {
      font-family: 'Segoe UI', Arial, sans-serif;
      color: #1e293b;
      margin: 0;
      padding: 20px;
      background: #ffffff;
    }
    h1 {
      color: #0f172a;
      border-bottom: 2px solid #3b82f6;
      padding-bottom: 8px;
      font-size: 24px;
    }
    h2 {
      color: #1e3a8a;
      margin-top: 24px;
      font-size: 18px;
    }
    .subtitle {
      color: #64748b;
      font-size: 14px;
      margin-bottom: 20px;
    }
    .diagram-container {
      background: #f8fafc;
      border: 1px solid #e2e8f0;
      border-radius: 8px;
      padding: 16px;
      margin-bottom: 30px;
      text-align: center;
    }
    table {
      width: 100%;
      border-collapse: collapse;
```

נתיב קובץ: temp_part_0.html | סיומת: .html | שורות קוד: 266

תפקיד הקובץ במערכת: רכיב UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body><!DOCTYPE html>
<html lang="he" dir="rtl">
<head>
  <meta charset="UTF-8">
  <title><title><title> UI_alfa Brokerage Platform</title>
  <style>
    @page {
      size: A4;
      margin: 18mm 15mm 18mm 15mm;
      @bottom-center {
        content: "עמוד " counter(page);
        font-family: sans-serif;
        font-size: 9pt;
        color: #64748b;
      }
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      color: #0f172a;
      line-height: 1.55;
      background-color: #ffffff;
      margin: 0;
      padding: 0;
      font-size: 10pt;
    }

    /* Cover Page - BLOCK layout, NO flexbox bugs */
    .cover-page {
      page-break-after: always;
      text-align: center;
      border: 3px solid #0284c7;
      padding: 50px 30px;
      border-radius: 12px;
      background-color: #f0f9ff;
      margin-bottom: 30px;
    }

    .cover-title {
      font-size: 26pt;
      font-weight: 800;
      color: #0f172a;
      margin-top: 20px;
      margin-bottom: 10px;
    }
  }
}
```

נתיב קובץ: temp_part_1.html | סיומת: .html | שורות קוד: 13

תפקיד הקובץ במערכת: רכיב UI, React Hook, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body>

  <!-- Chapter 1 -->
  <h1 id="chap1">סקירה ארכיטקטונית ומנגנוני הליבה</h1>
  <p>
    אז' אלגוריתמית מקצה לקצה. המערכת כוללת מנוע מסחר בזמן אמת, מנוע אשראי ומינוף <strong>UI_alfa</strong> מערכת
  </p>

  <div class="info-box">
    <strong>ארכיטקטורה מודולרית</strong> Backend-ה ב Java 17-תוכנן ב Spring Boot 3, וב
  </div>

</body></html>
```

נתיב קובץ: temp_part_10.html | סיומת: .html | שורות קוד: 304

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```

<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body>
  <!-- Chapter 10: Repositories -->
  <h1 id="chap10">פרק 10: מילון מפורט (13 מחלקות)</h1>

  <div class="class-card">
    <div class="class-title">com.brokerui.broker.AccountTransactionRepository</div>
    <div class="class-meta">נתיב: backend/backend/src/main/java/com/brokerui/broker/AccountTransa
    <p><strong>תפקיד המאגר במערכת:</strong> Spring Data JPA Repository מסד הנתונים מול שאילתות מול מסד הנתונים
    <p><strong>שאימות מותאמות:</strong> שאילתות ושיטות מותאמות
    <pre><code>package com.brokerui.broker;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AccountTransactionRepository extends JpaRepository<AccountTransaction, Long> {
  List<AccountTransaction> findByTradingAccountIdOrderByCreatedAtDesc(Long tradingAccountId);
}
</code></pre>
    </div>

    <div class="class-card">
      <div class="class-title">com.brokerui.broker.AuditLogRepository</div>
      <div class="class-meta">נתיב: backend/backend/src/main/java/com/brokerui/broker/AuditLogRepos
      <p><strong>תפקיד המאגר במערכת:</strong> Spring Data JPA Repository מסד הנתונים מול שאילתות מול מסד הנתונים
      <p><strong>שאימות מותאמות:</strong> שאילתות ושיטות מותאמות
      <pre><code>package com.brokerui.broker;

import org.springframework.data.jpa.repository.JpaRepository;

public interface AuditLogRepository extends JpaRepository<AuditLog, Long> {}
</code></pre>
    </div>

    <div class="class-card">
      <div class="class-title">com.brokerui.broker.BrokerOrderRepository</div>
      <div class="class-meta">נתיב: backend/backend/src/main/java/com/brokerui/broker/BrokerOrderRe
      <p><strong>תפקיד המאגר במערכת:</strong> Spring Data JPA Repository מסד הנתונים מול שאילתות מול מסד הנתונים
      <p><strong>שאימות מותאמות:</strong> שאילתות ושיטות מותאמות
      <pre><code>package com.brokerui.broker;

import java.util.List;
import org.springframework.data.jpa.repository.JpaRepository;

```

נתיב קובץ: temp_part_11.html | סיומת: .html | שורות קוד: 2447

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: `getUserProfile()`, `saveUserProfile()`, `useNotificationsWs()`, `fmtMoney()`, `ToastProvider()`, `BackPageHeader()`, `NNAdvisorPanel()`, `displayOrDash()`, `askQuestion()`, `fmtPrice()`, `xsrHeader()`, `SkeletonCard()`

```

<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body>
  <!-- Chapter 11: Frontend Files -->
  <h1 id="chap11">פרק 11: מילון מפורט - Frontend Components, Hooks & Scripts (46 קבצים)</h1>

  <div class="class-card">
    <div class="class-title">AdminDashboard_Local.html</div>
    <div class="class-meta">נתיב: AdminDashboard_Local.html | סיומת: .html | 1181 שורות</div>
    <p><strong>קובץ ניהול מצב, React Hook, UI רכיב תפקיד הקובץ במערכת</strong></p>

    <p><strong>פונקציות/רכיבים בקובץ</strong> <code>renderTradesTable()</code>, <code>mt5Connect()</code></p>
    <pre><code><!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Broker Admin | Local Controller</title>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&display=swap"
  <style>
    :root {
      --bg: #0a0c10;
      --bg-gradient: radial-gradient(circle at top right, #131a26 0%, #0a0c10 100%);
      --card-bg: rgba(13, 17, 23, 0.7);
      --card-shadow: 0 8px 24px rgba(0, 0, 0, 0.4);
      --primary: #00c8ff;
      --primary-glow: rgba(0, 200, 255, 0.4);
      --text: #f0f6fc;
      --text-muted: #8b949e;
      --border: rgba(255, 255, 255, 0.1);
      --success: #23d160;
      --danger: #f85149;
      --warning: #e3b341;
      --sidebar-w: 260px;
    }

    * { box-sizing: border-box; }
    body {
      margin: 0;
      font-family: 'Outfit', sans-serif;
      background: var(--bg-gradient);
      background-attachment: fixed;
      color: var(--text);
      display: flex;
      height: 100vh;

```

נתיב קובץ: temp_part_2.html | סיומת: .html | שורות קוד: 13

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body>

  <!-- Chapter 2 -->
  <h1 id="chap2">משתמש דמו vs. פרק 2: מנגנון הזדהות, אבטחה וכניסה - משתמש רגיל</h1>
  <p>
    המערכת מפרידה בצורה מלאה בין שני סוגי משתמשים:
  </p>
  <ul>
    <li><strong>משתמש רגיל (Real Account):</strong><code>AuthApiController.java</code> - נדרש ברישום ב
    <li><strong>משתמש דמו (Demo Simulators):</strong><code>AuthBootstrap</code> - מועלה אוטומטית בהפעלת השרת ב
  </ul>

</body></html>
```

נתיב קובץ: temp_part_3.html | סיומת: .html | שורות קוד: 12

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body>

  <!-- Chapter 3 -->
  <h1 id="chap3">פרק 3: מנגנון קנייה ומכירה של נכסים וביצוע הוראות</h1>
  <p>
    MARKET, LIMIT ו-STOP מנהל הוראות <code>OrderExecutionService.java</code> - מנוע המסחר ב
  </p>
  <div class="formula-box">
    Margin Required = (Execution Price × Quantity × Contract Size) / Leverage
  </div>

</body></html>
```

נתיב קובץ: temp_part_4.html | סיומת: .html | שורות קוד: 12

תפקיד הקובץ במערכת: רכיב UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body>

  <!-- Chapter 4 -->
  <h1 id="chap4">פרק 4: מנגנון הקרדיט ליין, ניהול סיכונים ומנוע הנזלה כפוייה</h1>
  <p>
    רוחים, (`tryCoverShortfall`) מעניק אשראי אוטומטי <code>MarginLoanService.java</code> -מנגנון האשראי ב
  </p>
  <div class="formula-box">
    Margin Level = Live Equity / Borrowed Balance
  </div>

</body></html>
```

נתיב קובץ: temp_part_5.html | סיומת: .html | שורות קוד: 9

תפקיד הקובץ במערכת: רכיב UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body>

  <!-- Chapter 5 -->
  <h1 id="chap5">MetaTrader 5 וגשר AI פרק 5: מנגנון המאצ'ינג, ניתוב</h1>
  <p>
    <code>NNPredictorClient.java</code> מנוע הניתוב משלב את B-Book vs. A-Book, להכרעת MT5 וגשר,
  </p>

</body></html>
```

נתיב קובץ: temp_part_6.html | סיומת: .html | שורות קוד: 455

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

◀ ▶

נתיב קובץ: temp_part_7.html | סיומת: .html | שורות קוד: 1059

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```

<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body>
  <!-- Chapter 7: Services -->
  <h1 id="chap7">מפרט: 7 פרק - Core Services & Integrations (15 מחלקות)</h1>

  <div class="class-card">
    <div class="class-title">com.brokerui.broker.MT5BridgeService</div>
    <div class="class-meta">נתיב: MT5_Infrastructure_v2/Backend/MT5BridgeService.java | סוג: class
    <p><strong>תפקיד המחלקה במערכת</strong> (Service / Client). ניתוב, אשראי, ניתוב סיכונים,
    <p><strong>פונקציות ושיטות במחלקה</strong> <code>isMT5Symbol()</code>, <code>getCandles()</code>
    <p><strong>תיאור דוקומנטציה</strong> STANDALONE MT5 BRIDGE INFRASTRUCTURE
    This service handles the communication with MT5 for:
    1. Chart Data (Candles)
    2. Real-time Price
    3. Order Execution</p>
    <pre><code>package com.brokerui.broker;

import org.json.JSONArray;
import org.json.JSONObject;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.net.InetSocketAddress;
import java.net.Socket;
import java.nio.file.Files;
import java.nio.file.StandardOpenOption;
import java.time.LocalDateTime;
import java.time.ZoneOffset;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.util.Locale;
import java.util.concurrent.TimeUnit;

/**
 * STANDALONE MT5 BRIDGE INFRASTRUCTURE
 * This service handles the communication with MT5 for:
 * 1. Chart Data (Candles)
 * 2. Real-time Price
 * 3. Order Execution
 */
@Service

```

נתיב קובץ: temp_part_8.html | סיומת: .html | שורות קוד: 366

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```

<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body>
  <!-- Chapter 8: Configs -->
  <h1 id="chap8">מפרט:8 פרק - Configs, Security & Setup (6 מחלקות)</h1>

  <div class="class-card">
    <div class="class-title">com.brokerui.BackendApplication</div>
    <div class="class-meta">נתיב: backend/backend/src/main/java/com/brokerui/BackendApplication.j
    <p><strong>תפקיד המחלקה במערכת</strong> Spring Boot, הגדרת קונפיגורציות Spring Security, אבטחת הגנות
    <p><strong>קונפיגורציות Spring Beans / Bootstrapping</strong> פונקציות ושיטות במחלקה</p>
    <pre><code>package com.brokerui;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

/**
 * The main application launcher class for the Spring Boot application.
 */
@SpringBootApplication
@EnableScheduling
public class BackendApplication {
  /**
   * The entry point of the Spring Boot JVM application.
   *
   * @param args command line arguments
   */
  public static void main(String[] args) {
    SpringApplication.run(BackendApplication.class, args);
  }
}
</code></pre>
    </div>

    <div class="class-card">
      <div class="class-title">com.brokerui.auth.AppUserDetailsService</div>
      <div class="class-meta">נתיב: backend/backend/src/main/java/com/brokerui/auth/AppUserDetailsS
      <p><strong>תפקיד המחלקה במערכת</strong> Spring Boot, הגדרת קונפיגורציות Spring Security, אבטחת הגנות
      <p><strong>פונקציות ושיטות במחלקה</strong> <code>loadUserByUsername()</code></p>
      <pre><code>package com.brokerui.auth;

import com.brokerui.user.AppUser;
import com.brokerui.user.AppUserRepository;
import java.util.List;
import java.util.Locale;
import org.springframework.security.core.authority.SimpleGrantedAuthority;

```

נתיב קובץ: temp_part_9.html | סיומת: .html | שורות קוד: 1202

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```

<!DOCTYPE html><html lang='he' dir='rtl'><head><meta charset='utf-8'></head><body>
  <!-- Chapter 9: Entities -->
  <h1 id="chap9">פרק 9: מילון מפורט (24 מחלקות) JPA Entities</h1>

  <div class="class-card">
    <div class="class-title">com.brokerui.auth.RegisterRequest</div>
    <div class="class-meta">נתיב: backend/backend/src/main/java/com/brokerui/auth/RegisterRequest
    <p><strong>תפקיד הישות במערכת:</strong> (JPA Entity / Data Model). ייצוג טבלת מסד נתונים (יומנים, וריציות, ודיווחים).</p>
    <pre><code>package com.brokerui.auth;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

public record RegisterRequest(
    @NotBlank @Email String email,
    @NotBlank @Size(max = 160) String displayName,
    @NotBlank @Size(min = 8, max = 72) String password) {}</code></pre>
    </div>

    <div class="class-card">
      <div class="class-title">com.brokerui.broker.AccountTransaction</div>
      <div class="class-meta">נתיב: backend/backend/src/main/java/com/brokerui/broker/AccountTransa
      <p><strong>תפקיד הישות במערכת:</strong> (JPA Entity / Data Model). ייצוג טבלת מסד נתונים (יומנים, וריציות, ודיווחים).</p>
      <pre><code>package com.brokerui.broker;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.FetchType;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import java.math.BigDecimal;
import java.time.Instant;
import org.hibernate.annotations.CreationTimestamp;

@Entity
@Table(name = "account_transaction")
public class AccountTransaction {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

```

נתיב קובץ: test_pages.html | סיומת: .html | שורות קוד: 526

תפקיד הקובץ במערכת: רכיב React Hook, UI, קובץ ניהול מצב (State Management), או תסריט עצמאי ב-Frontend.

פונקציות/רכיבים בקובץ: רכיב UI / תסריט סגנון

```
<!DOCTYPE html>
<html lang="he" dir="rtl">
<head>
  <meta charset="UTF-8">
  <title>Test Multi Page PDF</title>
  <style>
    @page {
      size: A4;
      margin: 15mm;
    }
    body {
      font-family: Arial, sans-serif;
      font-size: 10pt;
    }
    pre {
      background: #0f172a;
      color: #fff;
      padding: 10px;
      direction: rtl;
    }
  </style>
</head>
<body>

  <div style="page-break-before: always; padding: 20px; border: 1px solid #ccc; margin: 10px 0;">
    <h1>פרק 1 - כותרת מפורטת לבדיקת עמודים</h1>
    <p>העמוד נבנה ונבדק במנוע ההדפסה PDF-זוהי פסקת טקסט בעברית תקינה לבדיקת תהליך הרינדור של העמודים בקובץ הק>
    <pre><code>public class TestClass1 {
      public void executeAction() {
        System.out.println("Executing action for step 1");
      }
    }</code></pre>
  </div>

  <div style="page-break-before: always; padding: 20px; border: 1px solid #ccc; margin: 10px 0;">
    <h1>פרק 2 - כותרת מפורטת לבדיקת עמודים</h1>
    <p>העמוד נבנה ונבדק במנוע ההדפסה PDF-זוהי פסקת טקסט בעברית תקינה לבדיקת תהליך הרינדור של העמודים בקובץ הק>
    <pre><code>public class TestClass2 {
      public void executeAction() {
        System.out.println("Executing action for step 2");
      }
    }</code></pre>
  </div>

  <div style="page-break-before: always; padding: 20px; border: 1px solid #ccc; margin: 10px 0;">
```

